

APUNTESWEB · INGENIERÍA INFORMÁTICA

REDES

Segundo curso · 1.º cuatrimestre · 5 temas

JULIO DE 2026 · [HTTPS://ADRIANQL5.GITHUB.IO/APUNTESWEB/](https://adrianql5.github.io/apuntesweb/)

1. Introducción

Escrito por **Adrián Quiroga Linares**. [adriangl5](#)

1.1 Elementos de Internet

IPs

En Internet, cada dispositivo se identifica mediante una dirección *IP*.

- **IPv4** está formada por 4 bytes (32 bits) separados por puntos (.), por ejemplo, 192.168.0.1.
- **IPv6** está formada por 16 bytes (128 bits) separados por dos puntos (:), por ejemplo, 2001:0db8:85a3:0000:0000:8a2e:0370:7334.

No todas las **IPv4** tienen una **IPv6** asociada. IPv4 y IPv6 son sistemas de direccionamiento diferentes. Sin embargo, existen mecanismos de transición, como las direcciones IPv4-mapeadas en IPv6, que permiten la coexistencia.

Las direcciones IPv4 son **limitadas**, por lo que para paliar esta restricción, se emplean direcciones privadas y la técnica de **NAT** (Network Address Translation), que permite la traducción de direcciones privadas a públicas. Otra solución es la migración a **IPv6**, que ofrece un rango de direcciones **muchísimo mayor**.

Puertos

En un ordenador, cada aplicación se identifica mediante un **número de puerto**. Este número de puerto es un entero de 16 bits.

Sockets

Interfaz entre una **app** y la **capa de transporte**. Los sockets se contruyen con **IP y puerto**. Las direcciones IP serían como la dirección de un edificio, y los puertos indican el piso y letra. Los sockets son los buzones, y se identifican por el número de socket. Para 2 procesos situados en diferentes ordenadores necesitamos saber:

- La dirección **IP** del *ordenador en el cual se ejecuta el proceso*
- EL **puerto que tiene asignado el proceso dentro del ordenador**

Servidor y Cliente

El servidor es el conjunto de programa y computador que proporciona un servicio, el cliente es lo mismo solo que es el que solicita el servicio del servidor.

Hosts (estaciones/hospedadores)

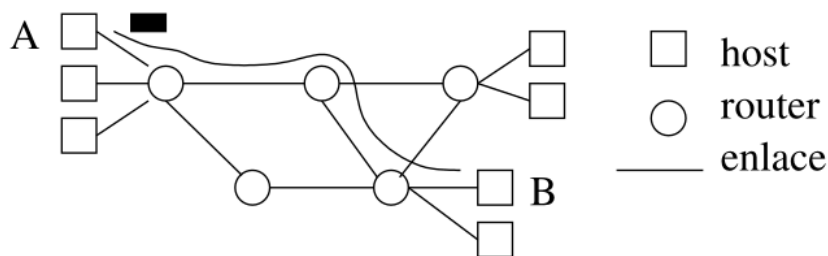
Son los sistemas terminales, el origen o el destino de las transmisiones. Incluyen a cualquier dispositivo capaz de conectarse a Internet.

Enlaces

Son los medios físicos por los que se realizan las transmisiones. Cables o transmisiones de radio.

Routers (rutadores o encaminadores)

Dispositivos que interconectan los enlaces. Los datos llegan al router por uno de los enlaces y este los reenvía a través de otros. Así tenemos una ruta entre origen y destino



Protocolos

Internet funciona porque todos sus componentes ejecutan los mismos protocolos, es decir, siguen las mismas reglas y el mismo formato para las comunicaciones. Distinguimos 2 tipos de protocolos:

- **Protocolos básicos:** necesarios para que funcione Internet (*TCP/IP*)
- **Protocolos de aplicación:** necesarios para que funcionen ciertas apps. Como *HTTP* (*web*), **SMTP** (*correo*), etc.

Proveedores de Internet (ISP)

Internet está operado por *numerosas compañías*. En el caso de Internet, distinguimos los siguientes proveedores de servicios de Internet (**ISP**):

- **Proveedores de baja escala (residenciales):** Son los que proporcionan acceso a Internet a los usuarios. El usuario hace **contrato** con ellos y luego parte del dinero se distribuye entre los demás proveedores.
- **Proveedores de alta escala (nacionales o internacionales):** Proporcionan los troncales que interconectan a los proveedores de baja escala y también las líneas de larga distancia.

1.2 Servicios orientado a Conexión y sin Conexión

Los servicios orientados a conexión y sin conexión en redes se refieren a cómo se maneja la comunicación entre dos dispositivos. Cada uno tiene características que lo hacen más adecuado para diferentes tipos de aplicaciones.

1.2.1 Servicio Orientado a Conexión

El protocolo **TCP (Transmission Control Protocol)** es el principal responsable de proporcionar un servicio confiable y orientado a conexión en Internet. Este servicio sigue tres fases:

Fases

1. **Establecimiento de la conexión:** Antes de transmitir datos, el cliente solicita al servidor establecer una conexión. Esta conexión se realiza mediante el famoso *handshake* de tres pasos (*SYN*, *SYN-ACK*, *ACK*).
2. **Transmisión de datos:** Una vez establecida la conexión, los datos se transmiten de manera confiable entre cliente y servidor.
3. **Desconexión:** Cuando la comunicación finaliza, ambos extremos cierran la conexión y liberan los recursos asignados.

A pesar de que hablamos de una *conexión*, esta conexión es a nivel de **software**, no de hardware, y afecta únicamente a los hosts implicados. Los routers intermedios no son conscientes de esta *conexión*. Esto significa que el *estado* de la conexión solo se mantiene en los dispositivos que están enviando y recibiendo datos.

Características

1. **Segmentación:** Si los datos enviados por la aplicación son grandes, **TCP** los divide en fragmentos más pequeños llamados *segmentos*. Estos segmentos se envían por separado y luego se reensamblan en el destino.
2. **Transferencia confiable:** TCP asegura que todos los datos lleguen sin errores y en el orden correcto. Para lograr esto, el receptor envía **confirmaciones (ACKs)** de los paquetes recibidos. Si el emisor no recibe una confirmación, asume que el paquete se perdió o llegó con errores, y lo retransmite.
3. **Control de Flujo:** Este mecanismo asegura que el emisor no envíe más datos de los que el receptor puede manejar. Si el receptor es lento o tiene poca memoria, puede reducir la cantidad

de datos que el emisor envía.

4. **Control de congestión:** TCP ajusta la cantidad de datos que envía en función de la capacidad de la red. Si detecta que la red está congestionada (es decir, muchos paquetes se pierden), reduce su tasa de envío para evitar saturar los enlaces y routers.

1.2.2 Servicio Sin Conexión

El protocolo **UDP (User Datagram Protocol)** ofrece un servicio **sin conexión**. Aquí, el emisor envía los paquetes de datos **sin necesidad de establecer una conexión** con el receptor. No hay fase de conexión, ni confirmaciones, lo que implica que:

1. **No hay garantías de entrega:** El emisor no sabe si los datos llegaron correctamente al destino, ya que no hay mecanismos para confirmarlo.
2. **No hay control de flujo ni congestión:** El emisor envía datos tan rápido como lo desee, sin considerar la capacidad del receptor o el estado de la red. Esto lo hace menos confiable, pero mucho más rápido que TCP.

Características de UDP

- **Simplicidad y rapidez:** UDP no se preocupa por establecer una conexión ni por la retransmisión de paquetes perdidos, lo que lo hace más rápido.
- **Menos fiabilidad:** No garantiza que los datos lleguen en el orden correcto o que lleguen sin errores.

Ejemplos de uso de UDP

UDP es ideal para aplicaciones donde la **velocidad** es más importante que la fiabilidad, como:

- **Videoconferencias:** Donde perder algunos paquetes de datos no afecta gravemente a la calidad de la llamada.
- **Juegos en línea:** La prioridad es la rapidez en la transmisión, y no necesariamente que todos los paquetes lleguen.
- **Streaming de medios:** Donde pequeños fallos o retrasos no son críticos.

Resumen

- **TCP:** Es ideal cuando la **fiabilidad** es crucial, pero es más lento debido al control de flujo, la retransmisión de datos y la gestión de conexiones.
- **UDP:** Es preferido para aplicaciones que requieren **rapidez** y pueden tolerar la pérdida de algunos datos, como las videollamadas o juegos en tiempo real.

1.3 Redes de Conmutación de Circuitos (CC)

En una red de este tipo, antes de transmitir los datos hay una fase de conexión en la que se **reservan los recursos hardware** que se usaran. Estos quedan reservados y no pueden ser utilizados por otra transmisión. Durante la **fase de conexión** se establece la **ruta**. La **transmisión** finaliza con una fase de **desconexión** que libera a todos los recursos.

Tenemos los siguientes circuitos conmutados:

- **Sin multiplexado:** por cada enlace solo se puede realizar **una transmisión** de cada vez.
- **Con multiplexado:** la capacidad de enlace **se reparte** entre varias transmisiones, dividiendo en varias ranuras el **tiempo** o la **frecuencia**. Cada ranura queda reservada a una determinada transmisión y las demás no las pueden usar.

1.3.1 Multiplexado por División en Frecuencia (DFM).

Por un **mismo enlace** se pueden enviar varias transmisiones si **modulamos** a una frecuencia distinta cada una de ellas. Por ejemplo las transmisiones por radio. En los **receptores**, las distintas **frecuencias** se pueden separar mediante **filtros**:

1.3.2 Multiplexado por división en tiempo (TDM)

Se asigna **periódicamente** una **ranura de tiempo** a cada transmisión. Una vez completado un ciclo (*marco*) se comienza de nuevo.

Las **redes de conmutación de circuitos** son **derrochadoras** puesto que reservan recursos aunque no los vayan a usar.

1.4 Redes de conmutación de paquetes (CP)

No se reservan recursos para la transmisión, todos se comparten y asignan bajo demanda. Estas redes trabajan con **paquetes**, que son bloques de datos de longitud determinada. Si el mensaje es más grande, se **segmenta** en varios paquetes. Cada paquete además de datos, contiene una **cabecera** con **información de control** para que el paquete pueda llegar al destino. Hay también paquetes que llevan **solo información de control**, como los **paquetes de confirmación**.

Los **routers** funcionan como **conmutadores de paquetes**, usualmente operando mediante almacenamiento y reenvío. Cuando el paquete llega al router, se procesa y almacena en la **cola de salida** hasta que le llega el turno para pasar al enlace (*siguiente router*). **Si la cola se llena, se descarta el paquete**.

1.4.1 Retardo

Componentes del **retardo**:

- **Retardo de procesamiento (router)**: examinar la cabecera y dirigir el paquete a la salida.
- **Retardo de espera en cola(router)**: proporcional a lo cargada que esté la red
- **Retardo de transmisión (router/host)**: tiempo que pasa entre que el host escribe en el enlace el primer bit del paquete hasta el último
- **Retardo de propagación(enlace)**: depende del tipo y longitud del enlace.

El retardo total se puede calcular como:

$$\text{Retardo_total} = \text{Retardo_procesamiento} + \text{Retardo_esperaEnCola} + \text{Retardo_transmisio}$$

Donde el **retardo de transmisión** del router/host se calcula como:

$$\text{Retardo_trans} = \frac{L}{R}$$

- L : tamaño del paquete (bits).
- R : velocidad de transmisión del router/host (bits por segundo).

Y el **retardo de propagación** del enlace se calcula como:

$$\text{Retardo_prop} = \frac{d}{V}$$

- d : distancia del enlace (metros).
- V : velocidad de propagación en el enlace (metros por segundo).

[! Info] El **ancho de banda** (BW) es una medida de la capacidad de transmisión de datos en una red o en un medio de comunicación. Básicamente, representa la **cantidad de datos que pueden enviarse o recibirse en un determinado período de tiempo**, generalmente medido en bits por segundo (bps) o sus múltiplos, como megabits por segundo (Mbps) o gigabits por segundo (Gbps).

En una red de 100 Mbps, el ancho de banda máximo es de 100 megabits por segundo. Si se supera este límite, habrá congestión, y los datos pueden retrasarse o perderse. Se puede pensar como **número de bits** que caben en el enlace. ****Capacidad de enlace= retardo * BW ****

1.4.2 Comparación de CC contra CP

Pregunta. Sea una red compartida por varios usuarios cada uno de los cuales transmite un 10 % del tiempo. (a) Si la red funciona en modo de **CC** con 10 canales (*ranuras de tiempo o frecuencias*), ¿cuántos usuarios podrá haber simultáneamente? (b) ¿Y si la red funciona en modo de **CP**? Respuesta. En la red de **CC** hay 10 canales, por lo tanto sólo puede haber 10 usuarios simultáneamente. Se supone

que la red tiene recursos para que cada uno de estos 10 usuarios transmita el 100 % del tiempo, aunque al final sólo lo haga el 10 % del mismo. Si la misma red (con la misma capacidad de routers y enlaces) se pone a trabajar en modo de CP, teóricamente podría soportar $10 \times 10 = 100$ usuarios simultáneamente. Sin embargo, esto es en teoría, porque en la práctica los retardos serían muy altos. En la práctica (*con cálculos de la teoría de colas*) podría haber 35 usuarios con retardos prácticamente despreciables.

1.4.3 Segmentación

Razones para la **segmentación**:

- **Menor tiempo de transmisión**
- Una **transmisión** no satura la red con mensajes enormes, sino que da oportunidad a otras transmisiones para que **intercalen** sus paquetes.
- En caso de **errores** en un paquete, solo hay que transmitir el paquete con errores.

1.4.4 Redes de Datagramas

En las **redes de datagramas** cada paquete se maneja de forma independiente. No se establece ninguna conexión antes de que los datos sean enviados.

Cada paquete tiene en su **cabecera** la dirección **IP de destino**. Cuando llega a un router, este lee la dirección y decide por qué camino enviarlo, colocándolo en la **cola de salida más adecuada**. Esto se conoce como **encaminamiento** y se usa una **tabla de rutas** para decidir a qué cola se debe enviar el paquete. Estas tablas se actualizan continuamente para reflejar los cambios en la red. Los routers además **no almacenan ninguna información** sobre los paquetes anteriores, por lo que paquetes de **la misma transmisión** pueden seguir rutas **diferentes** y podrían llegar al destino desordenados.

Si se cae un enlace de la red, los paquetes pueden ser **redirigidos** por otras rutas, sin embargo, no hay garantía de que los paquetes lleguen en el **orden correcto** o que todos lleguen. Son ideales para redes **grandes y heterogéneas** como **Internet**, donde es difícil planificar una ruta fija debido a la complejidad de la red.

1.4.5 Redes de Circuitos Virtuales

En las **redes de circuitos virtuales**, se establece una **conexión lógica** entre el emisor y el receptor antes de que los paquetes se transmitan. Esta conexión no es física, sino una **ruta planificada** que se sigue durante la transmisión.

Se necesita una **solicitud de conexión** por parte del **host origen** antes de comenzar a enviar datos, tras esto, la red **planifica una ruta** hacia el destino y asigna un **número de circuito virtual** a esa ruta. Los paquetes van a llevar ese **número de circuito virtual en la cabecera**, y los routers usarán ese número para dirigir a los paquetes, por lo que todos los paquetes de la **misma transmisión** seguirán el

mismo camino (a diferencia de la red de datagramas que iban todos a su bola). Finalmente se produce la **solicitud de desconexión** y se liberan los recursos asociados a esa transmisión.

El número de **circuito virtual** puede cambiar de un enlace a otro, lo que permite **reutilizar números de circuitos virtuales** en diferentes enlaces, lo que ahorra espacio en las cabeceras de los paquetes y nos evita **tener que mantener una tabla global** con todos los circuitos virtuales de la red. Cada router solo necesita saber qué números de circuito son válidos en los enlaces locales.

Ofrece una **ruta fija** para todos los paquetes, por lo que **llegan en orden y no se pierden**. Es más fiable para ciertas aplicaciones. Sin embargo, no es tan flexible como las redes de datagramas. Si algún enlace en la ruta falla, **toda la transmisión se ve afectada**. Son más adecuadas para redes más **pequeñas y controladas** donde se requiere mayor fiabilidad y velocidad en las transmisiones.

1.4.6 Comparación

Característica	Redes de Datagramas	Redes de Circuitos Virtuales
Encaminamiento	Independiente para cada paquete	Fijo, basado en el circuito virtual
Almacenamiento en routers	No se almacena información de paquetes anteriores	Se mantienen tablas de circuitos virtuales
Rutas	Pueden variar para cada paquete	Fija para toda la transmisión
Orden de llegada	Los paquetes pueden llegar desordenados	Los paquetes llegan en orden
Fiabilidad	Menos fiable	Más fiable

****A PARTIR DE AQUÍ NO SUELE CAER NADA MÁS DE ESTE TEMA.** IR AL 1.8**

1.5 Redes de difusión

Todos los **host** reciben las transmisiones pero solo la procesa el **destinatario**. La **wifi** es un ejemplo. Internet es una red de **datagramas**, así lo especifica **IP (TCP/IP)**. Sin embargo algunas capas de internet funcionan con otro tipo de tecnología: **conmutación de circuitos, circuitos virtuales o difusión**.

1.6 Acceso a Internet

El acceso **residencial a Internet** se puede hacer de 4 formas:

- **Módem telefónico**
- **ADSL**(línea digital asimétrica del suscriptor)

- **Cable HFC** (*híbrido fibra-coaxial*)
- **FTTH** (*fibra hasta el domicilio*) El **acceso empresarial** se suele hacer mediante redes locales *Ethernet, Wifi*.

1.6.1 Módem Telefónico

Usa la **línea telefónica** como si la transmisión a Internet fuese una llamada de voz normal. Primero llama al número telefónico del **ISP**(*Internet Service Provider*) y una vez establece la conexión convierte la señal digital en una analógica modulada. Esto se llama **modulación**:

EL **receptor** realiza la operación contraria, **demodulación**. **Módem**: **modular-demodular**. El problema es que las señales de voz que usa el sistema telefónico tienen **ancho de banda de frecuencia estrecha**. Como este ancho de banda solo puede conseguirse una velocidad de 50 kbps mediante modulación, era muy lento. Se conectaba el ordenador a la línea telefónica y las ondas transportaban la información.

1.6.2 ADSL

Línea digital asimétrica del suscriptor. Trata de **aprovechar todo el ancho de banda** en frecuencias del cable que conecta nuestra casa con la **central telefónica**. Hay que tener en cuenta que en las llamadas la capacidad de este cable es desaprovechada (*solo usamos los primeros 4KHz del ancho de banda aprox de 1MHz*):

Usa **multiplexión por división en frecuencia**, dividiendo el ancho de banda en 3 canales. EL primero transmite la voz telefónica, el segundo para el envío de Internet (*subida*) y el tercero para la recepción(*bajada*). El de **bajada es el de mayor velocidad**, porque el usuario **descarga más de lo que sube**.

Se pueden transmitir hasta 10 Mbps. Permite usar **teléfono y conexión a Internet por separado**.

1.6.3 Cable HFC (híbrido y coaxial)

La **cabecera final** es equivalente a la central telefónica, está centraliza todas las transmisiones de abonados que desde aquí pasan a Internet a través de un router. De aquí los **troncales de fibra** conectan a los nodos de fibra que a su vez mediante cable coaxial para dar servicios de **TV, Teléfono e Internet**. Las troncales se hacen a partir de fibra porque necesitan mucha capacidad, el resto de **línea coaxial** (*barato y fácil de instalar*). Todos los cables son multiplexados (*TDM y FDM*).

1.6.4 FTTH

Fiber to home:

- Fibra para distribución de servicios avanzados: *Triple Play*
- **OLT** (Optical Line Terminal): punto final que viene del ISP
- **ODN** (Optical Distribution Network): desde el OLT a los usuarios
- **ONT** (Optical Network Termination): conversión de señales ópticas ↔ eléctricas

1.6.5 Acceso Empresarial

Mediante una **LAN** conectada aun router y a un **ISP** con enlace dedicado

Red de la USC

- Tres nodos troncales en **Santiago** y uno en **Lugo** unidos a través de **RedIRIS Nova** y el **CESGA**: - Nodos troncales de Santiago unidos con enlaces dobles de 40 GE -Enlaces dobles desde los nodos troncales a los nodos de distribución de 10 GE -Enlaces a 10 GE entre los nodos de distribución y los nodos de acceso (conmutadores a 100 Mbps o 1 Gbps)
- Acceso a Internet mediante un nodo en el Cesga que enlaza con RedIris (gestiona la red pública)

1.7 Medios de Transmisión

Distinguimos entre:

- **Medios Guiados: Cable de par trenzado, Cable Coaxial y Fibra Óptica.**
- **Medios no Guiados: Atmósfera y espacio, canales de radio terrestre y canales de radio vía satélite.**

1.7.1 Cable de Par Trenzado

El **par trenzado (TP)** es el cable de tupo telefónico, Consta de 2 hilos de **cobre trenzado**, es decir, dispuestos en espiral y enrollados entre sí. En enrollado **disminuye alfo de pérdidas** de energía por radiación e interferencias. Dentro de un conducto de plástico pueden disponerse **cientos** de estos pares. Tenemos 2 tipos:

En el **apantallado (STP)**, que es de mayor calidad, el par esta recubierto de una película de metal con el objetivo de disminuir perdidas El par **sin apantallar (UTP)**, carece de este recubrimientos. Las categoría 1 y 2 son las de peor calidad y se usan en telefonía, y las de categoría 3 se usan en redes locales de 10Mbps y la de categoría 5 en las de 100Mbps.

1.7.2 Cable Coaxial

El cable **coaxial** esta formador por dos conductores: uno central sólido y otro exterior formador por una **mallla de hilos finos** que recubre al primero. Entre ambos hay plástico, la ventaja de esta disposición es que la radiación queda atrapada entre ambos y no escapa al espacio, **evitando pérdidas**.

Hay 2 calidades, el de **50 ohmios** que se utiliza para transmitir señales digitales en banda base (*sin modular*) en algunas redes locales. Y también el de **75 ohmios** de mejor calidad, se utiliza en banda ancha (*con modulación*) en las redes de **cable CGH** y permite cientos de canales de **TV, teléfono e Internet**.

1.5.3 Fibra Óptica

En vez de trasmitir señales eléctricas, **transmite luz**, evitando pérdidas por radiación. Las fibras se hacer a partir de vidrio o plástico transparente y ultrapuro. Tienen baja **atenuación**, por lo que pueden usarse hasta distancias de 100km **sin repetidores**. Son finas como un cabello, por lo que dentro de un conducto de pueden alojar miles

Precio elevado, Capacidad de Transmisión Mayor que cualquier Cable.

Fibra Multimodo

- La luz se propaga rebotando en las paredes del núcleo
- Utilizada para redes de conexión locales, centros de datos de edificio a edificio y para FTTH.

Fibra Monomodo

- Se propaga en línea recta $\Rightarrow$ mayor distancia
- Más costosa

Designación OC-n

Velocidad de enlace $n \times 51,8$ Mbps

****NO CREO Q PREGUNTE ESTO PERO ESTÁ BIEN PA ENTENDER EL CONCEPTO EN GENERAL DE LOS SIGUIENTES TEMAS.****

1.8 Arquitectura en Capas

La **arquitectura en capas** se utiliza para facilitar el diseño de los protocolos de comunicación. Consiste en dividir la comunicación en **tareas independientes** y poner cada una de ellas en una capa. Las **capas superiores** utilizan los servicios de las inferiores. La arquitectura en capas permite **conectar**

computadores y sistemas muy diferentes con la única condición de que respeten las especificaciones de cada capa. **Cada capa es independiente de las demás. Aunque las transmisiones son en vertical, los protocolos pueden diseñarse como si fuesen horizontales.** Cada capa puede ignorar el contenido de las capas contiguas, añadiendo instrucciones para el destinatario de cada capa.

En la **Capa de Aplicación**, se localizan los procesos que se comunican entre sí mediante mensajes. Por ejemplo si queremos comunicar 2 procesos (*un servidor con un cliente*). Para que se pueda realizar hay que acordar un **protocolo**: formato para los mensajes y unas reglas. Los datos se llaman **mensajes**.

La **Capa de Transporte** aparece ante la situación de que los procesos a comunicar están en **diferentes computadores** y que existe algún tipo de conexión entre estos. La **Capa de Transporte** prepara los mensajes para que puedan transmitirse fuera del computadores. Los datos se dividen en **segmentos** (en TCP) o permanecen como mensajes (en UDP).

La **capa de Red** aparece si los ordenadores **no están directamente conectados**, si no que forman parte de una red. Esta se encarga de buscar las rutas y llevar los paquetes desde **Host origen** hasta el **Host destino**. Los segmentos o mensajes se encapsulan en **paquetes o datagramas**.

La **Capa de Enlace** se encarga de los **detalles de bajo nivel** de la transmisión de cada bloque de datos (*paquete*) de un extremo a otro de un **enlace**. En este enlace se puede usar cualquier tipo de tecnología de red : *datagramas, circuitos virtuales, etc.* El enlace puede ser de **cable** o de **ondas** de radio y puede ser de **punto a punto**(*un emisor y un receptor*) o de **difusión**(*múltiples transmisiones*). Se encarga de comenzar la transmisión cuando el enlace está libre, de ir almacenando los datos en la memoria cuando van llegando, de detectar colisiones en enlaces de difusión, etc. Los paquetes o datagramas se convierten en **tramas** (frames).

La **Capa Física** trabaja a nivel de bits: convierte los bits **0 y 1** en **pulsos**, señales eléctricas. Conformar el pulso dándole la correspondiente forma, tensión y duración. También define las características mecánicas y físicas del medio de transmisión.

2. Capa de Aplicación

Escrito por **Adrián Quiroga Linares**.

2.1 Introducción

La capa de aplicación se ocupa de la **comunicación entre procesos**. El esquema típico se muestra en la siguiente figura:

RECORDATORIO

La comunicación de dos procesos, tanto en el **mismo host** como en hosts diferentes, se hace **enviando mensajes**. Para que los mensajes sean comprendidos hay que establecer un **protocolo**. El **protocolo** facilita la programación. Hay que programar tanto las funciones de envío como las de recepción y si tenemos un protocolo claro de programación será más fácil.

Se deben especificar que **tipos de mensajes** hay. Las **reglas** que especifican cuando se **envían o responden** mensajes. La **sintaxis del mensaje** y la **semántica de cada campo**.

Los protocolos de la **capa de aplicación** utilizan por debajo los protocolos básicos proporcionados por la capa de transporte. Como vimos en el tema anterior, estos son: **TCP y UDP**, TCP proporciona un servicio **fiable** y UDP un servicio **sencillo y rapido, pero no garantiza la recepción**.

Protocolo	Aplicaciones
TCP	HTTP (web)
	SMTP, POP3 (correo)
	FTP (ficheros)
UDP	DNS (traducciones)

NOTA

Denominamos **agente de usuario**, a la interfaz entre el usuario y la aplicación. O sea el programa gráfico que permite al usuario trabajar cómodamente con la aplicación, por ejemplo **firefox**.

2.2 Protocolo de Transferencia de Hipertexto HTTP

Este protocolo define la comunicación entre un servidor y un cliente web. Usa **TCP** y el por defecto, el **puerto 80**. Una **página web** usualmente consta de varios objetos, es decir, de varios **ficheros**:

Cuando un **navegador** solicita una **página web**, el primer objeto que se devuelve es el **documento base**, que contiene el layout de la página usualmente también el texto. El **layout** indica cómo conseguir lo restantes objetos y dónde han de colocarse.

030

Sin el **layout** no podemos saber como obtener el resto de objetos de la página web

2.2.1 Conexiones No Persistentes

Con **conexiones no persistentes** se utiliza una conexión **TCP** distinta para transferir cada uno de los objetos. Pueden ser o en **serie** o **paralelas**. Son en **serie** si se espera a que acabe la **conexión TCP** previa antes de que comience la siguiente. Son **paralelas** si se inician varias **conexiones TCP** a la vez.

2.2.2 Conexiones Persistentes

Con **conexiones persistentes** se utiliza la misma **conexión TCP** para conseguir **varios objetos** o incluso varias **páginas web** diferentes. Típicamente el **servidor** cierra la **conexión TCP** cuando no ha sido utilizada durante cierto tiempo. Existen dos versiones de **conexiones persistentes**, en la versión **sin entubamiento**, el cliente sólo pide un nuevo objeto cuando el previo ha sido recibido. En la versión **con entubamiento** el cliente puede hacer peticiones de varios objetos antes de recibir los anteriores. El modo por defecto de **HTTP/1.1** es **persistente con entubamiento**.

2.2.3 Tiempo de transferencia de una página web

El transferencia del primer objeto $2RTT + T_{transmission}$, esto se debe a que necesitamos conocer el **layout** de la página web para obtener el resto de objetos. En cambio, el **tiempo de transferencia** de los siguientes objetos depende del tipo de conexión.

- **Tiempo de ida y vuelta (*RTT round-trip time*)**: Es el tiempo que tarda una solicitud de red en ir de un punto de partida a uno de destino y volver al punto de partida. La solicitud no deja de ser un paquete muy pequeño. El *RTT* incluye los retardos de propagación de los paquetes, los retardos de cola en los routers y switches intermedios y los retardos de procesamiento de los paquetes.

- **Tiempo de transmisión del fichero:** tiempo que pasa entre que el host escribe en el enlace el primer bit del paquete hasta el último.

$$t_{\text{trans}} = \frac{L}{R}$$

- **Tiempo total para transmitir un paquete:**

$$\text{Tiempo total} = RTT + t_{\text{trans}}$$

IMPORTANTE

No se suman los tiempos extras del tema anterior, porque el **RTT** cuenta el tiempo de **propagación** puesto que no depende del número de bits a enviar, también cuenta el retraso en **colas** y el de **procesamiento**, tanto de ida como de vuelta.

2.2.4 Mensajes HTTP

El protocolo HTTP define cómo se envían y reciben mensajes entre un cliente (por ejemplo, un navegador web) y un servidor (el sitio web al que se accede). Estos mensajes se separan en **cabeceras** (headers) y un **cuerpo** (body), donde la cabecera contiene información de control y el cuerpo contiene los datos solicitados o enviados, como páginas web o imágenes.

Peticiones (Request): El cliente las envía al servidor para solicitar recursos, como páginas web o imágenes.

El significado de estos campos es el siguiente:

- **método:** GET (página normal) o POST (formulario)
- **sp:** espacio, en formato ASCII7
- **URL:** la dirección del objeto que se requiere
- **versión:** versión de HTTP en uso
- **cr:** retorno de carro, \r
- **lf:** salto de línea \n
- **nombreCampo:** puede haber varias líneas de este tipo. Esta variable puede ser:
 - Host (para el nombre del servidor)
 - Connection (que puede valer "close", para conexiones no persistentes)
 - User-agent (nombre del navegador en uso)
 - Accept-language (el idioma preferido por el solicitante)
- **valor:** para cada posible nombreCampo también tiene que especificarse el valor que toma
- **cuerpo:** en caso de que método=POST, contiene los datos del formulario

Esta estructura permite que el servidor entienda qué acción debe realizar, sobre qué recurso y con qué parámetros adicionales.

Mensaje de respuesta: El servidor responde con los recursos solicitados o con información sobre el éxito o fracaso de la petición.

El significado de estos campos es similar al de los mensajes de petición, pero:

- **cod:** es la codificación de la frase (por ejemplo, `200, OK`; `404, Not Found`; `400, Bad Request`)
- **nombreCampo:** ahora puede ser:
 - `Connection` (igual que en los de petición)
 - `Date` (fecha de envío)
 - `Server` (servidor que envía el objeto)
 - `Last-Modified` (fecha de la última modificación del objeto)
 - `Content-Length` (tamaño en bytes)
 - `Content-Type` (tipo de contenido del objeto, como una extensión)
- **cuerpo:** los datos del objeto que se envía

[! Info] Las **cookies** son pequeñas piezas de datos que un servidor web envía al navegador de un usuario y que el navegador guarda **localmente**. Cada vez que el usuario vuelve a visitar el sitio, el navegador incluye esas cookies en la solicitud HTTP, permitiendo que el servidor "recuerde" al usuario.

1. Cuando un usuario visita un sitio web, el servidor crea una **cookie** con un identificador único y se la envía al navegador.
2. El navegador guarda esta cookie en un **archivo de cookies**.
3. En visitas futuras, el navegador envía esta cookie al servidor junto con las solicitudes HTTP, lo que permite al servidor reconocer al usuario.

Cuando "Susana" visita Amazon, el servidor de Amazon le asigna un identificador (por ejemplo, "1678") y lo guarda en **su base de datos** junto con cualquier otra información relevante. Este ID se almacena en una cookie en el navegador de Susana. La próxima vez que Susana visite Amazon, su navegador enviará la cookie, y el servidor podrá identificarla y ofrecerle contenido personalizado.

Las cookies permiten a los sitios web identificar a los usuarios, recordar preferencias y mantener sesiones (por ejemplo, el carrito de compras en Amazon). Sin embargo, también plantean preocupaciones de privacidad, ya que permiten el seguimiento de la actividad del usuario en la web.

2.3 FTP: Protocolo Transferencia Ficheros

El **FTP (File Transfer Protocol)** es un protocolo diseñado para la **transferencia de archivos** entre un cliente y un servidor. A diferencia de otros protocolos, **FTP** utiliza **dos conexiones TCP en paralelo**: una para el **control** y otra para la **transferencia de datos**.

La **conexión de control** (*puerto 21*) es la que se utiliza para gestionar la sesión FTP, enviando y recibiendo **comandos** entre el cliente y el servidor. Este canal permanece **abierto durante toda la sesión**, es decir, **es persistente**.

- **Comandos transmitidos:** Se envían comandos como *nombre de usuario, contraseña, solicitar directorios, pedir archivos, cerrar la sesión, etc.*
- **Formato de los comandos:** Los comandos son legibles, utilizando texto ASCII de 7 bits, y se componen de palabras de 4 letras en mayúsculas seguidas de parámetros (si los hay). Ejemplo: `USER`, `PASS`, `LIST`, `RETR`.
- **Respuestas del servidor:** El servidor responde con **códigos de 3 dígitos** (que indican el resultado de los comandos) acompañados de una **explicación en texto**. Ejemplo: `230 User logged in, proceed.`

La conexión de control **se mantiene activa** durante toda la sesión de FTP, incluso cuando no se están transfiriendo datos.

La **conexión de datos** (*puerto 20*) se encarga de transferir los archivos o listas de directorios entre el cliente y el servidor. A diferencia de la conexión de control, esta conexión **no es persistente**, es decir, **se abre y se cierra** cada vez que se necesita transferir un archivo o realizar alguna operación que implique datos (como listar directorios).

- **Datos transmitidos:** Puede ser el contenido de los archivos que se están transfiriendo o el resultado de comandos como `LIST` (que devuelve un listado de archivos y directorios).
- **Formato de transmisión:** Los datos se transmiten en **binario**, lo que permite transferir cualquier tipo de archivo (texto, imágenes, vídeos, etc.) sin pérdida de información.
- **Apertura y cierre:** Cada vez que se necesita enviar o recibir un archivo, se establece una nueva conexión TCP para los datos y se cierra al finalizar la transferencia.

FTP es un protocolo con estado, lo que significa que el servidor mantiene información sobre la sesión mientras está activa. Entre los datos que guarda están:

- El **nombre del usuario** que ha iniciado sesión.
- El **directorio actual** en el que el usuario está navegando.
- Otros detalles que permiten al servidor recordar el contexto de la sesión hasta que el usuario cierra la conexión.

Esto es necesario para que el servidor pueda gestionar múltiples usuarios y archivos de forma ordenada.

Los **comandos FTP** que envía el cliente son cadenas de texto simples, normalmente de una **palabra de cuatro caracteres** en mayúsculas (como `USER`, `PASS`, `LIST`, `RETR`, `STOR`), y cada uno va acompañado de una acción específica. Por ejemplo:

- `USER <nombre>`: Enviar el nombre de usuario para autenticar.
- `PASS <clave>`: Enviar la contraseña.
- `LIST`: Listar los archivos del directorio actual.

- `RETR <archivo>`: Descargar un archivo desde el servidor.
- `STOR <archivo>`: Subir un archivo al servidor.

Las **respuestas del servidor** siguen un formato estándar de **códigos de tres dígitos**, que indican el estado de la operación, seguidos de un mensaje explicativo:

- `200`: Operación exitosa.
- `230`: Usuario autenticado correctamente.
- `550`: Error, archivo no encontrado o sin permisos.

Una de las limitaciones importantes de FTP es que **no cifra los datos**, incluyendo el **nombre de usuario y la contraseña** que se envían por la red. Esto significa que, en una red insegura, alguien podría interceptar y leer esa información.

2.4 SMTP: Protocolo Transferencia. Correo Sencillo

SMTP (Simple Mail Transfer Protocol) es el protocolo estándar para la transferencia de **correo electrónico** entre servidores. SMTP se encarga de **enviar correos** desde el cliente (el remitente) hacia un servidor de correo, y también entre servidores de correo.

NOTA

Solo se usa para enviar correos entre cliente-servidor y comunicar servidor-servidor. Para acceder al correo se usan protocolos más seguros como IMAP y POP3.

- **Puerto utilizado:** SMTP utiliza el **puerto 25** para comunicarse entre servidores de correo.
- **Conexiones TCP persistentes:** Al igual que FTP, SMTP mantiene una **conexión TCP persistente** mientras dure la sesión de envío del correo.
- **Formato de los mensajes:** Los correos y comandos que se envían a través de SMTP utilizan **ASCII de 7 bits**, por lo que los mensajes y las instrucciones son legibles por humanos.

El **sistema de correo electrónico** consta de varias partes:

1. **Agente de usuario (Mail User Agent, MUA):** Es el programa que los usuarios utilizan para gestionar su correo, como **Outlook**, **Thunderbird** o los clientes de correo de un navegador web (Gmail, Yahoo, etc.). El agente de usuario permite:
 - Escribir correos electrónicos.
 - Enviarlos a través de un servidor de correo.
 - Descargar correos recibidos desde un servidor, usando otros protocolos como **POP3**, **IMAP**, o incluso **HTTP** (en el caso de interfaces web).

Es importante notar que el **buzón de correo** (donde se almacenan los correos que recibes) no está en el ordenador local, sino en un **servidor remoto**. Esto es útil porque los correos pueden

llegar en cualquier momento del día, mientras que el ordenador del usuario no siempre está encendido o conectado a internet.

2. **Servidor de correo (Mail Transfer Agent, MTA):** Cuando el usuario envía un correo, su agente de usuario lo **transfiere al servidor de correo**. Este servidor de correo se encarga de **procesar** y **enviar** el correo al servidor de destino adecuado. Aquí es donde entra en juego **SMTP**.

El servidor de correo **origen** examina la dirección de destino y utiliza el protocolo **SMTP** para buscar el **servidor de correo destino**. Una vez identificado, transfiere el correo a dicho servidor, donde queda almacenado en el **buzón del destinatario**.

SMTP está diseñado para facilitar la **comunicación entre servidores de correo**. Aunque teóricamente la comunicación es directa entre dos servidores (origen y destino), en la práctica puede haber **servidores intermedios** que retransmitan los correos, especialmente si hay problemas de red o fallos temporales.

Para enviar un correo:

1. **El usuario escribe un correo** en su agente de usuario y lo envía.
2. **El agente de usuario** lo transfiere al servidor de correo origen.
3. **El servidor de correo origen** utiliza **SMTP** para establecer una conexión con el servidor de correo destino.
4. **SMTP** transmite el correo del servidor origen al servidor destino, donde el mensaje queda almacenado en el buzón del destinatario.
5. **El destinatario** puede, en cualquier momento, acceder a su correo y descargarlo mediante protocolos como **POP3**, **IMAP**, o directamente a través de un navegador web usando **HTTP**.

SMTP también maneja casos en los que no se puede entregar el correo de inmediato:

- Si el **servidor de correo destino** está **fuera de servicio** o no responde, el servidor origen pone el mensaje en una **cola de espera**.
- El servidor de correo intentará reenviar el correo cada cierto tiempo (generalmente, cada 30 minutos).
- Si después de varios intentos fallidos (normalmente, varios días) el correo no ha podido ser entregado, el servidor envía un **mensaje de error** de vuelta al remitente, informando que el correo no ha podido ser entregado.

SMTP (Simple Mail Transfer Protocol) utiliza tres tipos principales de mensajes para la comunicación entre el **cliente** (quien envía el correo) y el **servidor** (quien lo recibe):

- **Comandos**

- **HELO** (nombre del servidor)
- **MAIL FROM** (dirección del remitente)
- **RCPT TO** (dirección de destino)
- **DATA** (contenido)

- `QUIT`

- **Respuestas**

- Con códigos numéricos y frases aclaratorias

- **Datos**

- Son el contenido de los correos, con todos los objetos encapsulados

SMTP, tal como se diseñó originalmente, es un **protocolo inseguro** porque **no solicita autenticación del remitente**. Esto significa que cualquier cliente puede conectarse a un servidor SMTP y enviar correos a cualquier destinatario sin identificarse, lo que facilita la creación de **spam** (correos no deseados).

2.5 Protocolos de Acceso al Correo

Una vez que un correo ha llegado al **buzón de destino**, el siguiente paso es transferirlo al ordenador local del usuario. Para esto, no se utiliza SMTP, ya que este protocolo está diseñado para enviar correos (*es un protocolo de oferta*) y requeriría que el ordenador local estuviera siempre encendido y conectado a Internet, esperando la llegada de mensajes. Por lo tanto, se han diseñado protocolos específicos para **descargar correos**, siendo **POP3**, **IMAP** y **HTTP** los más comunes. Aquí nos centraremos en **POP3**.

2.5.1 Funcionamiento de POP3

POP3 (Post Office Protocol 3) permite a los usuarios descargar correos desde un servidor a su ordenador local. El proceso se basa en **tres tipos de mensajes** que se intercambian entre el **agente de usuario** (cliente) y el **servidor de correo**:

- **Comandos** (de 4 caracteres)

- `user` (nombre de usuario)
- `pass` (palabra clave)
- `list`
- `retr` (número de correo, para traer correo)
- `dele` (número de correo)
- `quit`

- **Respuestas**

- `OK` o `ERR`, con frases explicativas

- **Datos**

- La lista de mensajes, los contenidos de los correos, o el correo completo en un único mensaje

POP3 utiliza el **puerto 110** para la comunicación. Durante una sesión de POP3, se utiliza **una única conexión TCP persistente** que puede ser mantenida abierta mientras el usuario descarga varios correos. Esto ahorra tiempo y recursos en comparación con abrir y cerrar conexiones repetidamente. A diferencia de **SMTP**, **que permite enviar correos sin necesidad de una cuenta, POP3 requiere que el usuario tenga una cuenta y una contraseña válidas**. Esto significa que el **servidor verifica la identidad del usuario** antes de permitir el acceso a los correos, lo que ayuda a prevenir el acceso no autorizado.

Modos de Operación de POP3:

- **Descargar y borrar:** Con el comando `RETR` seguido de `DELE`, los correos son descargados y luego eliminados del servidor.
- **Descargar y mantener:** Solo se utiliza el comando `RETR`, permitiendo que los correos permanezcan en el servidor después de ser descargados.

POP3 es un protocolo eficiente y seguro para la transferencia de correos, permitiendo a los usuarios gestionar su correspondencia sin necesidad de mantener sus dispositivos siempre conectados. Su estructura de comandos y respuestas proporciona un método sencillo para acceder a los mensajes, lo que lo convierte en una opción popular para el acceso a correos en muchos entornos.

2.5.2 Funcionamiento de IMAP

IMAP es un protocolo diseñado para acceder y gestionar correos electrónicos de manera más **versátil y eficiente que su predecesor POP3**, especialmente para usuarios que necesitan **acceso remoto y sincronización constante entre dispositivos**.

Con POP3, los mensajes **se descargan y se guardan localmente en el dispositivo del usuario**. Una vez descargados, se **eliminan** del servidor (*aunque algunas configuraciones permiten dejarlos en el servidor*). Por lo que no se puede acceder a los mensajes desde otros dispositivos. Además, POP3 no soporta la creación y manejo de carpetas en el servidor remoto, lo que limita la organización del correo en múltiples dispositivos.

IMAP permite crear, renombrar y borrar carpetas **directamente en el servidor**. Los mensajes pueden moverse entre carpetas en el servidor. Esto asegura que la organización del correo sea consistente y accesible desde cualquier dispositivo conectado.

A diferencia de POP3, IMAP **guarda información sobre el estado de los mensajes y carpetas** en el servidor. Por ejemplo, si un mensaje ha sido leído, movido o borrado desde un dispositivo, esos cambios se reflejan en todos los dispositivos conectados.

El servidor asocia cada mensaje con una carpeta. Cuando llega un mensaje va a la carpeta **INBOX**. Permite leer, borrar, mover a otra carpeta. Hay comandos para crear carpetas, realizar búsquedas, descargar partes del mensaje.

2.6 DNS: Servicio de Nombres de Dominio

Nosotros en Internet, para poder comunicar a un cliente y a un servidor, necesitamos conocer ambos **sockets**. Si recordamos, los sockets están formados por **IP:Puerto**. Entonces, ¿cómo podemos conocer la IP destino de una página web? Para eso tenemos los **DNS**.

INFO

Con el comando `ping dominio` obtenemos la ip de un determinado dominio

El **Sistema de Nombres de Dominio (DNS)** es fundamental para el funcionamiento de Internet, ya que se encarga de traducir los **nombres de host** (como `www.ejemplo.com`) a **direcciones IP** (como `192.0.2.1`) y viceversa. Este sistema permite que los usuarios accedan a sitios web utilizando **nombres fáciles de recordar** en lugar de tener que memorizar largas cadenas de números.

DNS utiliza el protocolo **UDP** (*User Datagram Protocol*), específicamente el **puerto 53**. Esto se debe a que DNS no necesita mantener el estado de las conexiones. Cada host tiene una lista de direcciones de servidores de nombres, generalmente configurada durante la instalación del sistema operativo.

Cuando un host **necesita realizar una traducción**, se dirige al primer servidor de la lista. Si no recibe respuesta, se mueve al siguiente, y así sucesivamente. Esta estrategia permite que **DNS** sea más eficiente, ya que, en lugar de intentar reestablecer una conexión con el primer servidor, **busca otros**.

Realiza **traducciones** de nombres de hosts a direcciones IP y viceversa, permitiendo obtener alias de hosts. Indica **qué servidor está autorizado para proporcionar información** sobre un dominio específico, lo que es crucial para la administración de dominios.

Permite simplificar direcciones de correo electrónico. Por ejemplo, `usc.es` puede ser un alias para el servidor de correo `smtp.usc.es`, facilitando la escritura de direcciones como `peptio@usc.es`.

Un mismo nombre de host puede tener múltiples direcciones IP asociadas, lo cual es útil para **servidores espejo** (*servidores que tienen el mismo contenido*). DNS puede configurarse para devolver estas direcciones IP de manera rotativa.

2.6.1 Elementos del DNS

El DNS se compone de numerosos **servidores de nombres** distribuidos por Internet. Estos servidores trabajan en conjunto para **traducir** los nombres de dominio en direcciones IP.

El DNS utiliza un protocolo que permite a los hosts (computadoras) solicitar traducciones a los servidores de nombres y que **los servidores intercambien información entre ellos**.

Servidores Raíz: Hay un número limitado de servidores raíz en Internet que proporcionan información sobre los dominios de nivel superior (como `.com`, `.org`, `.es`). Si se pregunta a un servidor raíz sobre el dominio `.es`, proporcionará información sobre servidores de nombres en ese dominio.

Servidores Intermedios o TLD: Estos servidores proporcionan información sobre dominios de niveles intermedios (como `usc.es`). Si se le pregunta a un servidor en la raíz sobre el dominio `usc.es`, puede proporcionar información sobre un servidor en la Universidad de Santiago.

Servidores autoritativos: todas las organizaciones que tienen hosts accesibles públicamente a través de internet deben proporcionar registros DNS accesibles públicamente que establezcan la correspondencia entre los nombres de dichos hosts y sus direcciones IPs. Una organización puede elegir implementar su propio servidor DNS autoritativo para almacenar estos registros.

Servidores Locales: Son los que manejan las consultas de los hosts en una red local. Cada host tiene una lista de estos servidores, que a menudo son proporcionados por el ISP (Proveedor de Servicios de Internet). El host comienza consultando el primer servidor de la lista.

INFO

Un **dominio** es un nombre que identifica a un sitio web en Internet. Es una parte fundamental de la dirección de un sitio web y se utiliza para acceder a recursos en la red de manera fácil y comprensible para los usuarios. Por ejemplo, en la URL `www.ejemplo.com`, `ejemplo.com` es el dominio.

2.6.2 Tipos de Consultas a Servidores DNS

Consultas Recursivas: En este tipo de consulta, cada servidor de nombres se encarga de interrogar al siguiente hasta obtener la respuesta final.

Consultas Iterativas: En este caso, el servidor de nombres local se pone en contacto con todos los servidores necesarios para obtener la información.

2.6.3 Caché DNS

Cuando un servidor DNS obtiene una traducción, **almacena una copia en su memoria local** (disco o RAM) **para futuras consultas**. Esta caché se utiliza para responder a **consultas repetidas** de manera más rápida. Las entradas de la caché se eliminan después de un tiempo específico (generalmente, cada dos días).

2.6.4 Mensajes DNS

Existen **dos tipos de mensajes** en el DNS: **Consultas** y **Respuestas**. Ambos mensajes tienen un formato similar, que consta de una cabecera con información de control y un cuerpo que contiene las consultas y respuestas.

1. **Cabecera:** Contiene 6 campos, algunos de los cuales son en binario:

- **Identificación:** Un número de 16 bits que identifica la consulta, permitiendo que un host realice múltiples consultas simultáneamente.
- **Señales:** 4 bits que indican el tipo de mensaje (consulta o respuesta), si la respuesta proviene de un servidor autorizado, si se requiere una consulta recursiva, etc.
- **Tamaño de Campos del Cuerpo:** Indica la cantidad de preguntas, respuestas, servidores autorizados y datos adicionales en el mensaje.

2. **Cuerpo:** Consta de 4 campos:

- **Cuestiones:** Preguntas que incluyen el nombre o dirección IP a traducir.
- **Respuestas:** Respuestas obtenidas, como pares de nombres de host y direcciones IP. Un nombre de host puede corresponder a múltiples direcciones IP.
- **Servidores Autorizados:** Indica los servidores de nombres autorizados que se pueden consultar para un dominio específico.
- **Información Adicional:** Incluye datos que no fueron solicitados explícitamente.

2.7 Distribución de contenidos

La distribución de contenidos se refiere a las diversas **técnicas** utilizadas para gestionar y entregar contenidos digitales (*como páginas web, archivos de audio, video, etc.*) de manera eficiente, reduciendo el tiempo de carga y mejorando la experiencia del usuario.

Los accesos a servidores centralizados pueden ser lentos por la **congestión de red** y la **sobrecarga del servidor**. Para abordar estas limitaciones, se utilizan **métodos alternativos a los servidores centralizados**. En lugar de almacenar todos los contenidos en un solo servidor, se distribuyen y duplican en diferentes ubicaciones geográficas, dirigiendo las peticiones al servidor que **ofrezca el menor tiempo de respuesta**.

2.7.1 Caché Web (Servidor Proxy)

La **caché web** implica la utilización de un **servidor intermedio**, conocido como **proxy**, que gestiona todas las peticiones web de los usuarios en una red. Este servidor proxy normalmente es proporcionado por el **ISP**.

El navegador establece una **conexión TCP** con la **caché web** y envía una solicitud HTTP para el objeto a la caché web. La caché comprueba si tiene una **copia** del objeto almacenada localmente, si la tiene devuelve el objeto. Si no tiene el objeto, abre una **conexión TCP** con el servidor origen y envía una solicitud HTTP para obtener el objeto. Cuando recibe este objeto **almacena una copia localmente** y se lo envía como mensaje de respuesta HTTP al cliente.

Los usuarios deben configurar sus navegadores para utilizar el servidor proxy, indicando la dirección del mismo.

Puede haber un **esquema jerárquico de proxies**, donde varios proxies se comunican entre sí para optimizar aún más la distribución.

2.7.2 Redes de Distribución de Contenidos (CDN)

El objetivo es aumentar la velocidad y reducir la latencia. Las usan empresas que tienen páginas web muy visitadas (Youtube).

Las **CDN** son gestionadas por empresas que poseen una infraestructura de múltiples servidores distribuidos globalmente. Estas empresas no crean contenido, sino que alquilan su infraestructura a otras organizaciones. Por ejemplo, Akamai replica contenidos de sitios como El País en sus centros de hospedaje.

La **CDN replica los contenidos de sus clientes** en los servidores CDN y **los mantiene actualizados**. La CDN proporciona un mecanismo para que el contenido sea entregado por el servidor CDN que pueda hacerlo **más rápidamente**.

Cuando se indica al navegador del host de un usuario que extraiga un **vídeo** concreto, la CDN debe **interceptar** la solicitud para **determinar un conjunto de servidores** de la CDN que resulte **adecuado** para ese cliente en ese preciso instante y **redirigir** la solicitud del cliente a un servidor situado en dicho conjunto.

SI ESTAS EN ESPAÑA, NO TE VA A MANDAR A UN CONJUNTO DE SERVIDORES DE ETIOPIA POR EJEMPLO. SI TODO EL MUNDO ACCEDE A LOS SERVIDORES DE ISLANDIA, POSIBLEMENTE COLAPSEN.

2.7.3 Redes P2P (de Igual a Igual)

Las **redes P2P** permiten a los usuarios actuar tanto como clientes como servidores, compartiendo archivos entre sí. Al unirse a una red P2P, los usuarios pueden descargar y compartir contenido desde sus propios discos duros. Esto implica que no dependen de que haya un **servidor activo funcionando**. Un usuario se une a la red P2P ejecutando una aplicación (*bittorrent*).

Cuando se ejecuta la aplicación, lo primero que hace es buscar a otros usuarios P2P. Se necesita por lo menos un **nodo de arranque**, es decir por lo menos conocer a un usuario que ya esté en la red.

Una vez conecto a la red P2P, el usuario da a conocer **públicamente** el contenido de su disco duro. La red P2P necesita construir un **directorio** con la totalidad de los contenidos y su ubicación.

Directorio Centralizado

Napstar. Con este método, un único host en Internet centraliza la construcción y almacenamiento del directorio. Cuando un usuario ejecuta la aplicación P2P se dirige al **host central** y le proporciona su lista de contenidos. **El host central** construye el directorio con todos los datos proporcionados por los usuarios y atiende a todas las consultas.

Se puede pensar como que el **host central** contiene un listado de que ordenadores hay y qué archivos ofertan por eso es un directorio.

El directorio se **actualiza constantemente**. Cuando un usuario consigue un contenido pasa a ser servidor de ese contenido. Cuando un usuario deja de tener actividad es borrado del directorio.

Cuando se quiere obtener un archivo se consulta al **host central** y así sabe a que ordenador conectarse, porque el host central contiene toda la información de los ordenadores de la red.

Sin embargo, si falla el **host central** falla todo. El host central tiene que mantener el directorio de millones de usuarios y millones de consultas. **Hay un tráfico enorme hacia ese punto.**

Directorio Descentralizado

Kaaza .Consiste en distribuir el directorio entre algunos de los usuarios P2P. Ciertos usuarios son designados como líderes en función de ciertos criterios (*velocidad de las conexiones, antigüedad en la red, etc*).

Cada líder es responsable de atender las consultas de un grupo de usuarios que tiene a su cargo. Las consultas que no puede resolver por sí mismo las redirecciona a los otros líderes con los que está conectado. Esto origina una red de superposición (red virtual sobre la red real) con la correspondencia siguiente:

Se resuelven así el problema del **centralizado** que tenía un único punto de fallo y el cuello de botella.

Inundación de Consultas

Gnutella. Con este método, la red de superposición es plana, con todos los **usuarios al mismo nivel**. No se utiliza directorio, sino **inundación de consultas**. Las consultas **se retransmiten de usuario en usuario hasta que se encuentra el contenido solicitado**. El problema que tiene es que origina un tráfico enorme de mensajes. Esto se resuelve parcialmente limitando la profundidad de la inundación: se cuenta el número de nodos por los que pasa la consulta y cuando llegue a un cierto valor se da por terminada la consulta.

Descarga de Archivos

En una **aplicación P2P clásica**, una vez que un usuario ha descargado un fichero completo puede convertirse a su vez en **servidor de dicho fichero**.

Este método de descarga puede mejorarse si la descarga de los ficheros se hace por **segmentos**. Un usuario que ha descargado un segmento de un fichero puede convertirse a su vez en **servidor de dicho segmento** antes de que complete la descarga del fichero.

Podemos pensarlo como que cada usuario tiene fragmentos de 1 GB de una película pirata, y después si un usuario quiere verla, los junta en su ordenador. Van dpm pa piratear y no te pueden denunciar.

3. Capa de Transporte

Escrito por **Adrián Quiroga Linares**.

3.1 Introducción

La capa de transporte es un componente clave en la arquitectura de redes, cuya función es asegurar que los datos enviados desde una aplicación en el origen **lleguen de manera correcta** al proceso de destino.

RECORDATORIO

La **capa de transporte** se encarga de **preparar los datos para la transmisión**, recibe los **datos** que le pasa la **capa de aplicación** y los prepara para su transmisión dependiendo de si se utiliza **TCP o UDP**. Debe decidir si **fragmenta los datos en segmentos (TCP)** o simplemente les **añade una cabecera (UDP)**. Una vez los **datos llegan al destino** se debe encargar de reconstruir los mensaje que fueran segmentados y entregarlos al **proceso correcto** en el sistema receptor.

La capa de transporte **no necesita conocer** cómo **funciona el canal de transmisión**, ya que asume que puede haber muchos errores en la transmisión. El objetivo es realizar una **comunicación lógica** entre el origen y el destino, sin importar cómo se manejen los errores en las capas inferiores

La capa de transporte está presente solo en los dispositivos de origen y destino, no en los routers intermedios. Los routers se ocupan de las capas inferiores, como la red, pero no gestionan los detalles del transporte de datos.

TCP (Transmission Control Protocol): TCP divide los mensajes en **segmentos más pequeños**. A cada segmento le añade una **cabecera de control** para asegurar la correcta transmisión y el control de errores.

UDP (User Datagram Protocol): UDP **no realiza segmentación** de mensajes ni garantiza la entrega ordenada. Simplemente **añade una cabecera a los datos y los envía. Es más rápido, pero menos fiable que TCP**.

3.2 Multiplexión y demultiplexión

Los mensajes pasan de la capa de aplicación a la capa de transporte a través de un **socket**. Los procesos escriben y leen del socket. La capa de transporte recoge los mensajes del socket y los traslada al socket destino.

3.2.1 Multiplexión

La **multiplexión** es el proceso en el cual la capa de transporte recoge datos de múltiples **procesos de aplicación, los organiza y los envía a la capa de red**. Esto se hace recorriendo cada uno de los sockets asociados a las aplicaciones, procesando los datos de cada socket, y enviando los segmentos (*paquetes*) correspondientes a la red. Cada proceso de aplicación tiene su **propio socket**, lo cual permite que varios procesos diferentes transmitan datos simultáneamente. La multiplexión asegura que los datos de distintos procesos sean recogidos y enviados a través de la red de manera ordenada.

3.2.2 Demultiplexión

La **demultiplexión** ocurre en el lado del **receptor**. Aquí, la capa de transporte **recibe segmentos de la capa de red**, reconstruye los mensajes originales y los **distribuye a los sockets correspondientes** en función de los números de puerto que identifican a cada socket. De esta manera, los procesos en el lado del receptor pueden recoger los mensajes que les corresponden.

3.2.3 Identificación mediante puertos

Para identificar a qué socket corresponde un mensaje, se usan los **números de puerto**, que son enteros de 16 bits (0-65535). Estos números permiten diferenciar qué **proceso** debe recibir los datos en el sistema.

Los puertos entre 0-1023 están reservados para servicios bien conocidos (*como HTTP o FTP*), se conocen como **Puertos Conocidos**. Los puertos entre 1024-49152 están disponibles para las aplicaciones de usuario, se llaman **Puertos Registrados**.

Y los puertos restantes son los **Puertos Efímeros** que son los que se usan cuando por ejemplo abrimos varias pestañas de **firefox simultáneamente**, como no podemos usar el mismo puerto, el SO asigna a cada ventana un puerto efímero diferente, y al cerrarlas se liberan.

3.2.4 Multiplexión y Demultiplexión con sockets sin conexión (UDP)

En el caso de **UDP** (User Datagram Protocol), los sockets se identifican únicamente por la dirección **IP de destino y el puerto de destino**. Por ejemplo, un servidor que proporciona la hora del día a múltiples clientes (como el servicio daytime en el puerto 13) recibe datagramas de diferentes clientes y responde a cada uno de forma individual. Todos los datagramas se entregan al mismo puerto de destino, pero el puerto de origen permite que el servidor sepa a quién responder.

Si dos segmentos UDP tienen diferentes direcciones IP y/o números de puerto de origen, pero la misma dirección IP de destino y el mismo número puerto de destino, entonces los dos segmentos se enviarán al mismo proceso de destino a través del mismo socket de destino.

3.2.5 Multiplexión y Demultiplexión con sockets orientados a conexión (TCP)

En el caso de **TCP** (Transmission Control Protocol), los sockets se identifican con una tupla de 4 elementos: **la dirección IP de origen, la dirección IP de destino, el puerto de origen y el puerto de destino**. Por tanto, cuando un segmento TCP llega a un host procedente de la red, el host emplea los cuatro valores para **dirigir** (demultiplexar) el segmento al **socket apropiado**.

En particular, y al contrario de lo que ocurre con UDP, dos segmentos TCP entrantes con direcciones IP de origen o números de puerto de origen diferentes (con la excepción de un segmento TCP que transporte la solicitud original de establecimiento de conexión) **serán dirigidos a dos sockets distintos**. Cada conexión es atendida por un **proceso o hilo**.

En TCP, hay dos tipos de sockets:

1. **Socket de servidor**: Solo espera conexiones de clientes.
2. **Socket de conexión**: Se encarga de manejar una conexión específica.

Cuando un cliente establece una conexión con un servidor, el socket de servidor transfiere la conexión a un socket de conexión dedicado a esa interacción. Así, múltiples conexiones pueden ser manejadas simultáneamente por un solo servidor, como ocurre en servicios como **telnet** o **HTTP**.

Estos mecanismos permiten que múltiples aplicaciones en diferentes dispositivos puedan comunicarse de manera eficiente y ordenada a través de la red, utilizando un solo enlace físico pero gestionando múltiples flujos de datos simultáneos.

3.3 Capa de Transporte Sin Conexión: UDP

UDP (User Datagram Protocol) es un protocolo de transporte de capa 4 en el modelo OSI. A diferencia de TCP, es un protocolo **sin conexión**, lo que significa que **no establece una conexión entre el emisor y el receptor antes de enviar los datos, ni garantiza que los datos lleguen en orden o sin errores**. Es simple y poco sofisticado.

En el emisor, **UDP** coge el mensaje que se va a enviar y le añade una cabecera simple para formar un **segmento**. La cabecera UDP solo tiene cuatro campos (*puerto origen, puerto destino, longitud total y suma de comprobación*). La longitud total incluye el tamaño del segmento entero, contado la cabecera y datos. La **suma de comprobación** es un mecanismo para detectar errores en los datos. (*no se incluyen las IPs pero ya se verá mas adelante*.)

La suma de comprobación se usa para detectar errores en el segmento (tanto en los datos como en la cabecera). En el **origen, se suman todas las palabras de 16 bits del segmento**. Luego se realiza un complemento a 1 del resultado y se inserta en el campo de suma de comprobación. Al llegar al destino, se **recalcula** la suma de comprobación y **si coincide con la original, significa que el paquete llegó sin errores**.

En el receptor, **UDP** verifica si el segmento llegó sin errores utilizando la suma de comprobación. Si no hay errores, los datos se pasan al **socket** para ser procesados por la **aplicación**.

Es un servicio **sin conexión** puesto que no hay una negociación previa entre emisor y receptor. Los segmentos simplemente se envían sin establecer una conexión. Esto reduce la latencia ya que se evita el famoso **handshake** de TCP que se explicará más adelante. Sin embargo, **UDP** no garantiza la entrega de segmentos **ni los retransmite si se pierden o llegan con errores**.

Tiene un procesamiento rápido y bajo consumo de recursos, debido a que no mantiene el estado de la conexión ni tiene mecanismos de control de errores sofisticados, UDP es muy eficiente. Esto lo hace ideal para servidores que necesitan atender a muchos clientes simultáneamente o para aplicaciones que priorizan la velocidad.

La cabecera de UDP es mucho más pequeña que la de TCP, lo que implica un procesamiento **más rápido y menos sobrecarga de almacenamiento y transmisión**. Las aplicaciones que usan UDP pueden implementar sus propios mecanismos de control si es necesario, como la numeración de datos o la detección de errores adicional.

Ejemplos de usos de **UDP**:

- Traducción de nombres (*DNS*)
- Protocolos de encaminamiento (*RIP*)
- Administración de red (*SNMP*)
- Servidor de archivos remoto (*NFS*)

3.4 Fundamentos de la transmisión fiable

La transmisión fiable se refiere a la capacidad de garantizar que los datos enviados a través de una red lleguen correctamente al receptor. Para lograrlo, se utilizan **protocolos de retransmisión** conocidos como **ARQ (Automatic Repeat reQuest)**. Estos protocolos retransmiten paquetes si se detecta un error o si un paquete no llega al destino.

Las confirmaciones de recepción son los **ACKs** (*ACKnowledgment number*)

3.2.1 Protocolo ARQ: Parar y esperar

En este protocolo, el emisor envía un paquete y luego **se detiene a esperar una confirmación** (**ACK**) del receptor **antes de enviar el siguiente paquete**. Es un proceso simple pero **ineficiente** si hay mucha latencia o pérdida de paquetes.

Si no se producen errores, el receptor recibe el paquete correctamente y envía un **ACK** (*confirmación*) al emisor. **El número del ACK es el número del siguiente paquete esperado**. Una vez que el emisor recibe el ACK, envía el siguiente paquete.

Si se pierden paquetes, si un paquete se **pierde** en el camino o **llega con errores**, el receptor **no enviará el ACK**. El emisor tiene un **temporizador** que empieza a contar cuando se envía el paquete. Si no recibe el ACK antes de que **el temporizador expire, retransmite el paquete**.

****Si se pierde el ACK****, el emisor no sabrá que el receptor recibió el paquete. Entonces, cuando el ****temporizador** expire**, retransmitirá el paquete. El receptor, al recibir un paquete duplicado (*porque ya lo procesó antes*), simplemente reconoce que es un ****duplicado****, ****lo descarta**** y ****envía el ACK correspondiente nuevamente****.

Si el temporizador se configura con un tiempo muy corto o hay congestión en la red, tanto los paquetes como los ACK pueden retrasarse. El emisor puede recibir **duplicados de ACKs o de paquetes debido a este retraso**, pero los identifica por el número de secuencia y los **ignora si ya fueron procesados**.

NAK

Un **NAK** (*Negative-Acknowledge Character*) implica que si se recibe, el protocolo retransmite el último paquete y espera a recibir un mensaje ACK o NAK del receptor en respuesta al paquete de datos retransmitido.

Si el emisor recibe un ACK o NAK alterados **reenvía el paquete**. Sin embargo este método introduce **paquetes duplicados** en el canal emisor-receptor. La principal dificultad con los paquetes duplicados es que el receptor no sabe si el último paquete ACK o NAK recibido correctamente en el emisor. Por tanto no puede saber si un paquete entrante contiene datos nuevos o se trata de una retransmisión. Para solucionar esto surgen los **números de secuencia**.

Cuando recibe un paquete **fuera de secuencia**, el receptor envía un **paquete ACK para el paquete que ha recibido**. Cuando recibe un paquete corrompido, el receptor envía una respuesta de **reconocimiento negativo**. Podemos conseguir el mismo efecto que con una respuesta **NAK** si, **en lugar de enviar una NAK**, enviamos una respuesta de reconocimiento positivo (ACK) para el último paquete recibido correctamente. Un emisor que recibe **dos respuestas ACK para el mismo paquete** (es decir, recibe respuestas ACK duplicadas) sabe que el receptor **no ha recibido correctamente el paquete que sigue al que está siendo reconocido** (respuesta ACK) dos veces.

Sin embargo si vence el temporizador, se van a enviar **paquetes duplicados todo el tiempo**. Por ello debemos hacer que en vez de dos, **tres ACKs** equivalen a un **NAK**.

Inconveniente de parar y esperar

La **poca utilización de enlace**:

La **utilización del enlace** es la proporción del tiempo en que el emisor realmente está enviando datos, frente al tiempo total (que incluye tanto la transmisión como la espera). Es decir, es el **tiempo útil** dividido por el **tiempo total**. Se calcula de la siguiente forma:

$$U = \frac{t_{\text{trans}}}{RTT + t_{\text{trans}}}$$

3.2.2 ARQ con ventana deslizante (entubamiento) El protocolo **ARQ con ventana deslizante**, también conocido como **entubamiento**, es una mejora sobre el protocolo "parar y esperar", y su principal ventaja es que **reduce el tiempo perdido** esperando confirmaciones (*ACKs*). En lugar de enviar un paquete y esperar una confirmación antes de enviar el siguiente, el emisor puede enviar **varios paquetes consecutivos** (*hasta un número máximo N antes de recibir los ACKs*). Esto mejora considerablemente la **utilización del enlace**.

La **utilización del enlace** se mejora al permitir que se transmitan múltiples paquetes antes de recibir un ACK.

$$U = \frac{N \cdot t_{\text{trans}}}{RTT + t_{\text{trans}}}$$

La utilización es **máxima** cuando:

$$N \geq 1 + \frac{RTT}{t_{\text{trans}}}$$

La **ventana emisora** es el conjunto de 1 hasta N paquetes que el emisor **puede enviar antes de recibir una confirmación**. Estos paquetes pueden haber sido enviados, pero aún **no han sido reconocidos**.

La **ventana receptora** es el conjunto de hasta N paquetes que el receptor **puede aceptar y procesar, pero que aún no ha reconocido**.

El rango de **números de secuencia** debe abarcar al menos el **del tamaño de la ventana emisora**. Esto es necesario **para que no haya confusión entre paquetes viejos y nuevos que puedan usar el mismo número de secuencia**.

Tanto el emisor como el receptor **deben poder almacenar varios paquetes en memoria**. El emisor guarda los paquetes que **ha enviado pero aún no han sido reconocidos**, y el receptor guarda los paquetes que **ha recibido pero aún no ha procesado**.

El emisor envía varios paquetes de manera consecutiva sin esperar una confirmación por cada uno. La ventana del emisor se va desplazando conforme el emisor recibe los ACKs de los paquetes. Por ejemplo, si la ventana tiene un tamaño de $N = 5$, el emisor puede enviar 5 paquetes, pero la ventana solo avanzará cuando reciba confirmaciones de los paquetes. Cuando se recibe un ACK, la ventana **"desliza"** y permite que el **emisor envíe más paquetes**.

Retroceso N (Go-Back-N, GBN):

El receptor solo acepta paquetes en orden. Si un paquete se pierde o llega con errores, el receptor **descarta todos los paquetes que lleguen después del paquete problemático**. El emisor debe

reenviar no solo el paquete que falló, sino también todos los paquetes siguientes. Los ACKs son **acumulativos**, lo que significa que un ACK para un paquete también **confirma la recepción de todos los paquetes anteriores**. No se pueden enviar paquetes desordenados.

$N=4$

****Repetición selectiva:**** El receptor puede aceptar paquetes ****fuera de orden****. Si un paquete ****se pierde o llega con errores****, el receptor puede almacenar los paquetes posteriores y ****solo solicitar la retransmisión del paquete perdido****. Solo se retransmiten los ****paquetes que se pierden o llegan con errores****, lo que mejora la eficiencia en comparación con Go-Back-N. Se envía un ACK para cada paquete recibido correctamente, incluso si llegan fuera de orden.

$N = 4$

3.3 TCP

TCP (*Transmission Control Protocol*) es un protocolo que asegura una transmisión de datos fiable. TCP asigna **números de secuencia a los segmentos de datos**. Estos números de secuencia son de 32 bits y tienen algunas particularidades:

- **Inicialización aleatoria:** En lugar de comenzar en 0, TCP elige un número aleatorio (llamémoslo x) como punto de partida, lo que ayuda a diferenciar distintas conexiones y reducir posibles problemas de seguridad.
- **Incremento en función de los bytes enviados:** Cada segmento tiene su propio número de secuencia, que es igual al número del último byte del segmento anterior más el número de bytes del segmento actual.

En muchas comunicaciones, los datos se transmiten en ambas direcciones. TCP optimiza el envío de datos y reconocimientos (ACK) utilizando una técnica llamada **superposición o piggybacking**. Esto significa que cuando un segmento de datos llega al receptor, este puede esperar un momento a enviar el ACK, **combinándolo con los datos de respuesta para reducir el número de transmisiones**.

TCP utiliza **ACK acumulativos**, si el emisor envía varios segmentos, **un solo ACK puede confirmar múltiples segmentos**, lo cual es más eficiente. Si el emisor no recibe un ACK dentro del tiempo del temporizador, **solo retransmitirá el segmento específico no reconocido**, optimizando el uso de ancho de banda. Puede aceptar también **segmentos en desorden** y esperar a que lleguen después los intermedios.

Cuando el temporizador expira, **TCP retransmite el segmento y duplica el tiempo de espera del temporizador**. Este ajuste progresivo del temporizador ayuda a mejorar el rendimiento en redes con

retrasos fluctuantes.

NOTA

Esto de aquí abajo ni caso, si lo pone se la saca.

$$\text{Temporizador} = \text{EstimacionRTT} + 4 \times \text{DevRTT}$$

EstimacionRTT es una media ponderada de los RTT recientes (el tiempo entre enviar un segmento y recibir el ACK)

$\text{EstimacionRTT} = (1 - \alpha) \times \text{EstimacionRTT} + \alpha \times \text{MuestraRTT}$ **DevRTT** mide la variación de RTT, ayudando a TCP a ajustar los tiempos de espera.

$\text{DevRTT} = (1 - \beta) \times \text{DevRTT} + \beta | \text{MuestraRTT} - \text{EstimacionRTT} |$ Normalmente $\alpha = 0.125$ y $\beta = 0.25$

3.3.1 Detección de Fallos en TCP

TCP no tiene reconocimientos negativos (**NAK**) específicos, pero se interpretan los **ACK triplicados** como un **NAK** lo que implica reenviar el paquete.

Cuando el receptor recibe un segmento con mayor número al esperado, es decir, cuando se forma un agujero, el receptor envía un ACK duplicado. El emisor lo interpreta sólo como un aviso.

Si esto vuelve a pasar, se envía otro ACK repetido. El emisor recibe entonces **tres ACKs** repetidos y en este caso realiza una **retransmisión rápida**, en la cual retransmite el segmento ausente **antes de que venza el temporizador**.

Hay dos eventos principales de pérdida:

- **Vencimiento del temporizador:** se considera grave, porque se han perdido tanto los segmentos como los reconocimientos.
- **ACKs triplicados:** Este caso se considera leve, puesto que han llegado los reconocimientos.

El control de flujo es el mecanismo que permite al receptor indicar al emisor el **ritmo al que puede recibir datos**. En el momento de la conexión, el receptor indica el tamaño de su ventana de recepción y el emisor fija a este valor su ventana de envío. Esto se hace mediante un campo en la **cabecera TCP**, lo que permite modificar también este valor en cada transmisión.

TCP establece la **conexión** a través de el **three-way handshake** para sincronizar los números de secuencia y asegurarse de que ambas partes estén listas para intercambiar datos. La secuencia es la siguiente:

- **Emisor** → Solicitud de conexión (SYN), número de secuencia x .
- **Receptor** → Aceptación (SYN, ACK), número de secuencia y , reconoce $x + 1$.
- **Emisor** → Confirmación (ACK), reconoce $y + 1$.

Tras la confirmación, se pueden enviar datos en ambas direcciones.

Desconexión: La desconexión es un proceso en dos pasos:

- Cuando una de las partes desea finalizar la conexión, envía una solicitud de desconexión (FIN). La otra parte envía un ACK para reconocerla y también envía un FIN.
- Ambas partes cierran el flujo en la dirección opuesta.

3.3.2 Estructura del segmento TCP

Cuando enviamos, el proceso va escribiendo sus datos al **socket**, y las especificaciones de TCP indican que debe coger los datos del socket para transmitir según convenga.

TCP segmenta los datos y el tamaño máximo del segmento (*sin contar la cabecera*) se denomina **MSS** (*Maximun Segment Size*).

Los segmentos además se pueden **pushear** por petición de la aplicación de usuario (enviar el paquete aunque no llene el MSS). También se pueden especificar **datos urgentes** y también se puede **resetear la conexión**.

IMPORTANTE

Si por ejemplo dicen de calcular el número de bits que ocuparía el campo de número de secuencia. Hay que tener en cuenta el número de bytes máximo que pueden circular por la red durante el tiempo que vayan a estar vivos.

Por ejemplo si $TTL = 60S$ y $T_{trans} = 100 Mbps$ van a circular $6 \times 10^9 bits$ por lo que necesitamos $\frac{6 \times 10^9}{8}$ números de frecuencia.

Además el tamaño de la ventana receptora se calcula como : $CW = T_{trans} \times RTT$

La fórmula viene de la definición de ventana de recepción, donde es el tamaño máximo de paquetes que el receptor puede recibir y procesar pero que no confirmado. Se expresa en bytes.

Tiene **puerto origen** y **puerto destino**. **Número de secuencia** inicializado en un número aleatorio. **Número de reconocimiento**, número de bytes recibidos correctamente siempre que el bit **ACK** está activado. En caso de no estarlo, este campo no se usa. **Longitud de la cabecera**

Indicadores

- **URG** – El segmento tiene datos urgentes (urgent).
- **ACK** – El segmento tiene un reconocimiento (acknowledgment).
- **PSH** – El segmento fue empujado (push).
- **RST** – Se solicita reinicio de la conexión (reset).

- **SYN** – Se solicita inicio de conexión (synchronize).
- **FIN** – Se solicita fin de conexión (finished).

También contiene el **tamaño de la ventana de recepción**, el receptor le indica al emisor cuantos bytes puede recibir. Se usa para el control de flujo.

Suma de comprobación de 16 bits en complemento a uno (C1) de la cabecera TCP y cuerpo (y algunos campos más de la cabecera IP). Se calcula en los dos lados de la transferencia para verificar la transmisión.

Apuntador de datos urgentes, indica la posición de los datos urgentes dentro del segmento. En caso de que haya datos urgentes, **URG** está a 1. Y opciones, tiene varios usos, como negociar el tamaño máximo de los segmentos.

3.4 Congestion

3.4.1 Introducción

La congestión en redes TCP ocurre cuando **demasiados paquetes intentan atravesar la red al mismo tiempo**, causando retrasos y pérdida de datos debido a la saturación de las memorias de los routers. Esta saturación hace que los routers tengan que **desechar paquetes**, lo cual provoca más retransmisiones y empeora el problema. TCP implementa mecanismos para **manejar esta congestión**, regulando la tasa de envío en función de la congestión percibida en la red. Estos mecanismos son esenciales para mantener un flujo de datos eficiente y equilibrado. Veamos en detalle cada escenario de congestión y las técnicas de control que usa TCP:

3.4.2 Posibles Escenarios

Memoria Infinita en los Routers

Imaginemos que los routers pueden almacenar una cantidad infinita de datos. En este caso, conforme la tasa de envío de paquetes se acerca a la **capacidad del enlace**, los paquetes comienzan a acumularse en la cola del router, **generando retrasos**. Estos retrasos se van acumulando con cada nuevo paquete, creando un "cuello de botella".

Memoria Finita en un Router

Cuando la cola se llena, el router empieza a **descartar paquetes**. Esto obliga al emisor a **retransmitir** los paquetes perdidos, lo que, a su vez, **aumenta aún más la carga en la red y agrava la congestión**.

Múltiples Routers

Cuando hay varios routers en el camino, si un paquete **se pierde a mitad del camino**, los routers anteriores han gastado recursos procesando un paquete que finalmente no llegó a su destino. Esto reduce la eficiencia general de la red, ya que se desperdicia capacidad en procesar paquetes que no alcanzarán su destino.

Tasa de transmisión entre 0 y C/2: todo se recibe OK y con **retardo finito**.

3.4.3 Control de Congestión en TCP

En **TCP/IP**, el mecanismo de control de congestión recae en **TCP**, que considera que hay congestión cuando **expira un temporizador** o **se reciben ACKs triplicados**. El **emisor** disminuye la tasa de emisión cuando nota que hay congestión. TCP **define en el emisor** dos variables que va actualizando el **RTT** y la **ventana de congestión**.

La **ventana de congestión** es una variable controlada por el emisor que **limita la cantidad máxima de datos que pueden enviarse antes de recibir un reconocimiento (ACK)** (*Es lo mismo que la ventana receptora anteriormente mencionada*). Se puede entender como limita a la **ventana de emisión**. Su tamaño se ajusta **dinámicamente**, aumentando gradualmente cuando no hay pérdidas y reduciéndose cuando detecta congestión. Se mide en **bytes**.

$$\text{ventana de congestión} = T_{\text{trans}} \times \text{RTT}$$

EXPLICACIÓN

Esto se puede entender como que es el número de bytes que se pueden transmitir durante un RTT, que incluye la ida y vuelta de los paquetes.

También es útil esta fórmula:

$$T_{\text{trans}} = \frac{\text{ventana de congestión}}{\text{RTT}}$$

Inicio Lento

Al iniciar una conexión, TCP configura una **ventana de congestión inicial pequeña**, equivalente al tamaño máximo del segmento (**MSS**). La ventana de congestión se incrementa **exponencialmente**, duplicándose en cada intervalo igual al **RTT**, hasta que se detecta una pérdida de paquetes o la ventana alcanza el tamaño del umbral de congestión. Por ejemplo, si el **MSS** es de 500 bytes y el **RTT** es de 0.2 segundos, la tasa de transmisión inicial sería (*se mide en bits por segundo*):

$$T_{\text{trans}} = \frac{500 \times 8}{0.2} = 20 \text{ kbps.}$$

Cuando ocurre una pérdida o se alcanza el umbral, TCP abandona el inicio lento y pasa a la siguiente fase.

AIMD (Additive Increase, Multiplicative Decrease):

Se basa en dos principios:

Aumento aditivo cuando no hay congestión (es decir, no se detectan pérdidas de paquetes), la ventana de congestión aumenta de forma lineal (*en unidades de 1 segmento por RTT*).

Decremento multiplicativo cuando se detecta congestión (por ejemplo, a través de la pérdida de paquetes), la ventana de congestión se **reduce a la mitad** (una disminución multiplicativa).

INFO

Umbral de Congestión (ssthresh):

- **Definición:** El umbral de congestión es el valor que marca el límite entre las fases de **inicio lento** y **AIMD**.
- **Función:** Indica el tamaño de la ventana en el que TCP pasa de **inicio lento** a **AIMD**. En inicio lento, la ventana crece exponencialmente hasta alcanzar este umbral, luego se controla linealmente en AIMD.
- **Crecimiento:** El umbral se ajusta cuando se detecta una pérdida de paquete:
- **Si ocurre congestión** (pérdida de paquetes), el umbral se reduce a la mitad de la ventana de congestión, pasando a AIMD.

Vencimiento del Temporizador

Cuando el **temporizador de retransmisión expira** sin recibir un **ACK**, TCP interpreta esto como una congestión **grave**. Reinicia la ventana de congestión a **1 MSS**, volviendo a la **fase de inicio lento**.

Por ejemplo:

- Si la ventana de congestión era de **24 KB** antes del vencimiento:
 - Se reinicia a **1 MSS**.
 - Crece exponencialmente hasta **12 KB** (mitad del tamaño previo).
 - A partir de ahí, el crecimiento se vuelve lineal.

Recuperación Rápida en TCP

La **recuperación rápida** es un mecanismo dentro de TCP diseñado para mejorar la **recuperación de la ventana de congestión** después de una **pérdida de paquetes**.

A diferencia de un **fin de temporización**, que reinicia la ventana de congestión a 1 MSS, la **recuperación rápida** permite a TCP **recuperarse sin reducir drásticamente su ventana**, manteniendo la eficiencia de la conexión.

Cuando TCP recibe **tres ACKs duplicados**, interpreta esto como una señal de congestión leve (es decir, solo un paquete se ha perdido, pero la red no está completamente congestionada).

Reducción de la ventana de congestión: - La ventana de congestión se reduce a la mitad del tamaño previo, **más un margen de 3 MSS** (tamaño máximo de segmento). - La fórmula para calcular la nueva ventana de congestión es:

$$\text{Nueva ventana de congestión} = \frac{\text{Ventana previa}}{2} + 3 \text{ MSS}$$

Retransmisión del paquete perdido: Durante esta fase, TCP retransmite el segmento perdido y continúa enviando nuevos segmentos dentro del límite de la ventana ajustada.

Incremento lineal (AIMD): Una vez que se ha resuelto la congestión, la ventana de congestión se incrementa de forma **lineal** (en unidades de 1 segmento por RTT) hasta alcanzar el ancho de banda disponible sin causar sobrecarga en la red.

[! Info] En TCP distinguimos dos tipos de TCP:

- **Reno:** si recibimos 3 **ACKs** duplicados reduce la ventana a la mitad de su valor actual y comienza la fase de **recuperación rápida**.
- **Tahoe:** al recibir 3 **ACKs** duplicados reduce la ventana a **1 MSS** y vuelve a la fase de **inicio lento**. Reno es más eficiente. Si se produce **timeout (expira el contador)** ambos vuelven a la fase de **inicio lento**.

3.4.4 Imparcialidad

Cuando varias conexiones TCP **comparten un enlace**, ajustan su **tasa de envío automáticamente para repartir la capacidad disponible** del enlace de manera equitativa. Por ejemplo, si dos conexiones TCP comparten un enlace de 10 Mbps, cada una recibirá aproximadamente 5 Mbps.

Sin embargo, cuando se introduce una conexión **UDP** junto con TCP, la conexión UDP, al **no tener control de congestión**, puede **acaparar la mayor parte del ancho de banda**. Del mismo modo, si se establece una conexión TCP junto con nueve conexiones TCP paralelas, el enlace se repartirá en una proporción de 1/10 para la primera conexión y 9/10 para las demás.

4. Capa de Red

Escrito por **Adrián Quiroga Linares**.

NOTA

Tema jodido.

4.1 Introducción

La **capa de red** en el modelo de Internet es la encargada de transportar los datos desde un host de origen hasta un host de destino, controlando el **encaminamiento y el reenvío de paquetes** a través de la red. Se implementa tanto en los dispositivos **terminales** como en los **routers**.

RECORDATORIO

Host Origen, Host Destino y Router

El **host origen** es el que **recibe el paquete** de la capa de transporte, que encapsula los datos de la aplicación. Se encarga de **encapsular el paquete en un datagrama IP** que contiene la información necesaria para su entrega, como la dirección de origen y destino. **Entrega el datagrama en la capa de enlace**.

El **host destino** desencapsula el **datagrama** y entrega el segmento a la **capa de transporte**.

Los **routers** juegan un papel crucial en la capa de red. Aunque no interactúan directamente con las capas de transporte o aplicación (excepto para control o gestión de red), manejan el **reenvío y el encaminamiento de los datagramas** basándose en la dirección IP de destino y en su **tabla de reenvío**.

Reenvío, Encaminamiento y Tablas de reenvío

El **reenvío** se produce cuando un router recibe un **datagrama**, utiliza la dirección de destino en la **cabecera** del datagrama y consulta en su **tabla de reenvío** para decidir la **interfaz de salida adecuada**. El **encaminamiento** es el proceso de determinar la **mejor ruta** para los paquetes que se envían de un emisor a un receptor. Este proceso es dinámico y utiliza **algoritmos de encaminamiento** que construyen y actualizan las **tablas de reenvío** en cada router. Las **tablas de reenvío** almacenan la información necesaria para el reenvío de paquetes, indicando la interfaz de salida correcta para cada

destino. Los valores en estas tablas son determinados por los protocolos y algoritmos de encaminamiento.

****Control de Errores**

Suma de comprobación de la cabecera para detectar errores en la cabecera de un paquete IP, se utiliza una **suma de comprobación** que verifica la integridad de los datos. Si la suma de comprobación no coincide en el receptor, el paquete se descarta.

Seguridad: Originalmente, IP no tenía mecanismos de seguridad. Sin embargo, con el tiempo se han añadido tecnologías como IPsec, que permite la **autenticación y cifrado** del tráfico IP, transformando el servicio de conexión sin estado en uno con una capa de seguridad añadida.

La **capa de red** permite la transmisión de paquetes entre hosts a través de routers, usando técnicas de **encapsulación, reenvío y encaminamiento**. La red IP es no fiable y sin estado, proporcionando un servicio de **mejor esfuerzo**. Gracias a protocolos auxiliares como **ICMP, DHCP y NAT**, y mecanismos de control de errores y congestión, la capa de red es capaz de mantener la integridad y eficiencia del flujo de paquetes en redes de gran escala como Internet.

4.2 Redes de Conmutación de Paquetes

Las **redes de conmutación de paquetes** permiten transmitir datos dividiéndolos en **paquetes pequeños**. Estos paquetes viajan a través de la red de manera independiente, siguiendo rutas que pueden variar de un paquete a otro. El término "conmutación de paquetes" se refiere al proceso de enviar estos paquetes a través de varios nodos (por ejemplo, routers) que los redirigen hasta su destino.

4.2.1 Redes de Datagramas:

Una **red de datagramas** es un tipo de red de conmutación de paquetes donde los paquetes se envían de **manera independiente**, sin necesidad de establecer una ruta fija entre el origen y el destino antes de la transmisión. Cada paquete incluye la **dirección IP del destino en su cabecera**, lo que permite que los routers encaminen los paquetes de manera independiente según su propia **tabla de enrutamiento**.

En Internet, la capa de red se basa en el **Protocolo de Internet (IP)**, el cual utiliza este modelo de **datagramas**. Cada paquete contiene información suficiente en su **cabecera** (como la dirección IP de destino) para que los routers puedan determinar a dónde debe enviarse el paquete siguiente. Los **routers** en una red de datagramas no mantienen información sobre el estado de las conexiones previas o el tráfico pasado. Cada paquete es tratado como una entidad independiente, y el router no sabe nada sobre los paquetes que llegaron antes o después.

El servicio proporcionado por las redes de datagramas es de **mejor esfuerzo** (*best effort*). Esto implica que **no se garantiza la entrega de los paquetes, su orden ni el tiempo en que llegarán** al destino.

Los paquetes pueden llegar desordenados, con retraso o incluso perderse durante la transmisión debido a problemas como congestión en la red.

Los routers toman decisiones de encaminamiento en función de la dirección IP de destino contenida en la cabecera de cada paquete. Estas decisiones se toman consultando las **tablas de reenvío** de los routers, las cuales pueden actualizarse dinámicamente mediante protocolos de enrutamiento (como **OSPF** o **BGP**). La red de datagramas IP permite que diferentes tipos de redes de enlace de datos (Ethernet, Wi-Fi, 4G, etc.) se conecten entre sí, lo que facilita la **interconexión de redes heterogéneas**.

4.2.2 Redes de Circuitos Virtuales:

En contraste con las redes de datagramas, las **redes de circuitos virtuales** requieren la **planificación de una ruta fija** antes de que los datos puedan ser enviados. Este proceso implica la creación de un **circuito virtual (CV)**, que es una ruta preestablecida que todos los paquetes siguen durante la comunicación.

Antes de que los paquetes de datos sean enviados, se establece una conexión en la que los routers **planifican una ruta fija** para todo el flujo de paquetes entre el origen y el destino. Los paquetes de datos, una vez la ruta esté establecida, incluyen un **identificador de circuito virtual (VCI)** en la cabecera, que los routers utilizan para reencaminar los paquetes. A diferencia de las redes de datagramas, los routers en una red de circuitos virtuales mantienen **información de estado** acerca de las rutas y las conexiones activas. Cada router tiene una **tabla de circuitos virtuales** que almacena los identificadores de los circuitos virtuales y las interfaces de salida correspondientes.

La creación de un circuito virtual se inicia cuando el emisor, A, envía una **petición de llamada** al receptor, B. Esta solicitud llega primero al nodo 1 a través de la interfaz 2. En este punto, el nodo 1 asigna un **identificador de circuito virtual (VCI)**, que en este caso es 5, y reenvía la solicitud a través de su interfaz 1 hacia el siguiente nodo. Este proceso es similar al enrutamiento de datagramas, donde cada nodo debe saber cómo redirigir la solicitud hasta el destino final, B. A medida que la solicitud avanza: - El nodo 2 recibe la solicitud, le asigna un VCI de 9 y la reenvía por su interfaz 0. - El nodo 3 recibe la solicitud, le asigna un VCI de 7 y la envía por su interfaz 3. - Finalmente, el receptor B recibe la solicitud y asigna un VCI de 4, que usará para identificar los paquetes provenientes de A.

Una vez que B acepta la conexión, envía una **respuesta de llamada aceptada** de regreso a través del mismo camino pero en sentido inverso. Esta respuesta llega al nodo 3 con VCI 4 en la interfaz 3, permitiéndole completar su entrada en la tabla de circuitos. Del mismo modo, los nodos 2 y 1 actualizan sus tablas con los VCIs de salida correspondientes.

Finalmente, el nodo 1 envía una **confirmación de aceptación (ACK)** de vuelta a A, que la recibe con el VCI 5. Desde este momento, A usará el VCI 5 para identificar y enviar el resto de paquetes hacia B, estableciendo así el circuito virtual completo.

Una vez que la conexión ha sido establecida, los paquetes subsiguientes llevan el identificador del circuito (VCI). Los routers no necesitan volver a determinar la ruta, ya que esta ha sido preestablecida, y pueden utilizar el **VCI** para encontrar la interfaz de salida correcta.

Resumen Comparativo:

Característica	Redes de Datagramas	Redes de Circuitos Virtuales
Establecimiento de Ruta	Dinámico, cada paquete tiene su propia ruta.	Estática, se establece una ruta fija.
Estado de los Routers	Sin estado, no mantienen información de paquetes previos.	Con estado, mantienen información de las conexiones.
Identificador de Ruta	Basado en la dirección IP de destino.	Basado en un identificador de circuito virtual (VCI).
Confiabilidad	No fiable, no se garantiza la entrega, orden o tiempo.	Más fiable, pero depende de la ruta establecida.
Ejemplo	Enrutamiento de un paquete en Internet.	Conexión telefónica en una red de conmutación de circuitos.

Ambos modelos tienen aplicaciones diferentes según las necesidades de la red. Las **redes de datagramas** son más flexibles y escalables, lo que las hace ideales para Internet, mientras que las **redes de circuitos virtuales** proporcionan una calidad de servicio más consistente y son más adecuadas para aplicaciones que requieren un flujo continuo de datos, como las videollamadas o las comunicaciones en tiempo real.

4. 3 Algoritmos de Encaminamiento

Los **algoritmos de enrutamiento** son fundamentales para determinar el camino más eficiente que deben seguir los paquetes de datos desde un origen hasta un destino en una red. Estos algoritmos se basan en el concepto de **grafos**, donde los **routers** son los **nodos** y los **enlaces** entre ellos son las **aristas**. El coste de un enlace puede ser determinado por diversos factores, como la distancia, la velocidad, el nivel de congestión o incluso el coste económico del enlace.

4.3.1 Clasificación de los algoritmos de enrutamiento

1. **Algoritmos globales (de estado de los enlaces, EE):** Estos algoritmos requieren que cada nodo tenga **conocimiento completo** sobre la topología de la red, es decir, que conozca todos los enlaces y sus costos. Cada router utiliza esta información para calcular de manera **independiente** su **tabla de enrutamiento**, determinando el camino más corto hacia cada destino. El término "estado de los enlaces" hace referencia al hecho de que los nodos conocen el **estado de todos los enlaces**. Esto implica que el algoritmo realiza un proceso similar al de resolver un problema de **camino mínimo en un grafo ponderado**.

Ejemplo de algoritmo global: OSPF (Open Shortest Path First), que se utiliza en redes de área extensa (WAN) y utiliza el algoritmo de **Dijkstra** para encontrar la ruta más corta.

2. **Algoritmos descentralizados (de vector de distancias, VD):** En estos algoritmos, cada nodo **no tiene conocimiento completo** de la topología de la red. En cambio, **colabora** con los nodos vecinos intercambiando información de distancias. Inicialmente, cada nodo conoce únicamente las distancias a sus nodos vecinos. A través de un proceso iterativo, los nodos comparten y actualizan esta información hasta que todos los nodos tienen conocimiento de las distancias a otros nodos de la red. Este tipo de algoritmo se denomina **vector de distancias** porque cada nodo mantiene un vector con las distancias a los nodos vecinos.

Ejemplo de algoritmo de vector de distancias: RIP (Routing Information Protocol), que envía mensajes periódicos a los vecinos para actualizar la información de enrutamiento.

3. **Algoritmos estáticos y dinámicos: Estáticos:** Solo se actualizan cuando cambia la topología de la red (por ejemplo, si se agregan o eliminan nodos o enlaces) o si se realizan cambios manuales en los parámetros. **Dinámicos:** Se ejecutan de forma periódica y automática para adaptarse a cambios en la red, como modificaciones en la carga o el estado de los enlaces. Los algoritmos dinámicos son más comunes en Internet, ya que permiten adaptarse de forma continua a las variaciones en la red.

Ejemplo de algoritmo dinámico: BGP (Border Gateway Protocol), que es utilizado para el enrutamiento entre diferentes sistemas autónomos (AS) en Internet.

4. **Algoritmos sensibles a la carga e insensibles a la carga: Sensibles a la carga:** El coste de los enlaces cambia dinámicamente en función de la carga o congestión de la red. Esto permite que el algoritmo adapte los caminos de enrutamiento según el tráfico. **Insensibles a la carga:** El coste de los enlaces es constante y no cambia con la congestión de la red. La mayoría de los algoritmos de enrutamiento actuales en Internet son insensibles a la carga. **Problemas de oscilación:** Si un algoritmo es sensible a la carga, pueden ocurrir oscilaciones de enrutamiento, donde los caminos más cortos cambian de forma continua. Un enlace muy congestionado podría tener un coste elevado y ser evitado, pero luego, al liberar la congestión, se vuelve atractivo nuevamente, lo que puede generar fluctuaciones en el enrutamiento.

DE LOS ALGORITMOS SUDAR BTT. ****SALTAR AL 4.4****

4.3.2 Algoritmo de Dijkstra

El **algoritmo de Dijkstra** es un algoritmo de búsqueda de caminos mínimos, utilizado principalmente en redes de computadoras y grafos para encontrar el camino más corto desde un nodo de origen a todos los demás nodos en un grafo ponderado (con costes en los arcos o enlaces).

El enfoque de "**Forward Search**" del algoritmo de Dijkstra se basa en mantener dos listas: **Confirmado** y **Provisional**, para calcular la tabla de rutas de un nodo a partir de los paquetes de estado de enlace (LSP) recibidos. Cada nodo N quiere calcular su tabla de enrutamiento considerando los enlaces con otros nodos y sus costes. Tiene una complejidad de $O(n^2)$

Inicialización:

1. **Lista Confirmada:** Es la lista de nodos cuyo camino mínimo ya se ha encontrado. Se inicia con una entrada para el nodo de partida N con un coste de 0, indicando que el coste para llegar a sí mismo es cero. La entrada tiene el formato $(N, 0, -)$:
 - $(M, 5, L) \rightarrow$ N llega a M con coste 5 a través de L.
 - $-$ significa que no hay siguiente salto porque es el nodo de partida.
2. **Lista Provisional:** Es la lista de nodos que aún no han sido definitivamente asignados, pero cuyas distancias son conocidas parcialmente. Inicialmente está vacía.

Pasos del algoritmo:

1. **Seleccionar el Nodo S:** El algoritmo toma el último nodo añadido a la lista **Confirmado**. Este nodo S es el nodo del que se calcularán las distancias hacia sus nodos vecinos.
2. **Examinar los Vecinos de S:**
 - Para cada vecino V de S (según el LSP de S), el algoritmo calcula el coste para alcanzar V como la **suma del coste de N a S** (que es el coste ya conocido en **Confirmado**) y el **coste de S a V** (que se obtiene del LSP de S).
 - Si V no está en ninguna de las listas (ni **Confirmado** ni **Provisional**), se añade a la lista **Provisional** con la entrada (V, Coste, S) , donde **Coste** es la suma de las distancias y **S** es el siguiente salto.
 - Si V ya está en la lista **Provisional**, pero el nuevo coste calculado es menor que el coste previamente almacenado en esa lista, se reemplaza la entrada con el nuevo coste y el siguiente salto correspondiente, que es el nodo S.
3. **Actualizar la Lista Confirmado:**
 - Si la lista **Provisional** está vacía, el algoritmo termina, ya que no hay más nodos por explorar.
 - Si no está vacía, se selecciona el nodo de **Provisional** con el menor coste y se mueve a **Confirmado**, marcando ese nodo como definitivamente alcanzado con el coste mínimo.
4. **Repetir el Proceso:**
 - El proceso se repite desde el paso 1, pero ahora se toma el siguiente nodo con el menor coste en la lista **Provisional**. Este nodo se convierte en el nuevo nodo **S** y se repite el proceso de exploración y actualización.

4.3.3 Algoritmo de Vector de Distancias (VD)

El **encaminamiento de vector de distancias** es un método descentralizado de enrutamiento, donde cada nodo en una red colabora con sus vecinos para determinar la distancia mínima hacia todos los

demás nodos. Este método se basa en la propagación gradual de la información de distancias, permitiendo que cada nodo descubra rutas óptimas hacia todos los destinos mediante **iteraciones sucesivas**.

Funcionamiento del Algoritmo

1. Inicialización:

- Cada nodo conoce inicialmente solo el **coste directo** hacia sus vecinos inmediatos. Estos costes directos se almacenan en una **tabla de distancias**, donde cada entrada indica el nodo vecino y el coste del enlace hacia él.
- No tiene información sobre nodos más alejados en la red.

2. Intercambio de Información:

- Cada nodo comunica periódicamente a sus vecinos lo que sabe hasta ese momento sobre las distancias mínimas hacia los otros nodos.
- La información compartida es esencialmente el vector de distancias del nodo que envía el mensaje, es decir, sus estimaciones actuales de distancia hacia cada destino posible.
- Al recibir la información de un vecino, cada nodo calcula nuevas rutas y actualiza sus distancias si encuentra valores menores.

3. Actualización de Distancias:

- Para un nodo x con un vecino z , cuyo enlace tiene coste $c_{x,z}$:
 - Si z informa de que su distancia a otro nodo y es $d_{z,y}$, el nodo x puede calcular su distancia a y a través de z como:

$$D_{x,y}(z) = c_{x,z} + d_{z,y}$$

- Esto significa que la distancia de x a y a través de z es la suma del coste del enlace de x a z y la distancia de z a y reportada por z .
- Si el valor $D_{x,y}(z)$ es menor que la distancia almacenada actualmente en la tabla de distancias de x hacia y , x actualiza su distancia mínima hacia y y marca a z como el **siguiente salto** hacia y .

4. Iteración y Convergencia:

- Este proceso continúa iterativamente: cada nodo sigue intercambiando información con sus vecinos y recalculando rutas hasta que ya no se producen cambios en las tablas de distancias de los nodos.
- En ese momento, el algoritmo **converge**, y cada nodo ha encontrado el camino más corto hacia todos los otros nodos de la red.

El problema es que si un **nodo** calcula mal sus **distancias** jode a todos los demás, por lo que es poco robusto.

4.3.4 Encaminamiento Jerárquico

El **ruteo jerárquico** es una estrategia de enrutamiento utilizada en redes grandes, como Internet, para organizar y simplificar la gestión de los datos y mejorar la eficiencia del enrutamiento. En este enfoque, la red completa se divide en **sistemas autónomos (SA)**, que son zonas o regiones bajo un control administrativo común. Cada sistema autónomo (SA) puede pertenecer, por ejemplo, a una empresa o a una organización, y opera su propia red interna. Los routers **solo conocen el encaminamiento en su región**. Los routers pasarela frontera: centralizan el tráfico de salida del SA.

Hay dos niveles de encaminamiento:

- **Intradominio:** cada SA puede elegir algoritmo
- **Interdominio:** común para todos los SA

4.4 Encaminamiento en Internet (Protocolos de Rutado)

El encaminamiento en Internet se basa en protocolos de rutado, que **permiten a los routers decidir el mejor camino para enviar paquetes de datos a su destino**. Internet se divide en diferentes sistemas autónomos (SA), que son redes administradas de forma independiente. Estos sistemas usan distintos protocolos de rutado para decidir cómo enviar datos tanto dentro del **SA** (*rutado intra-autónomo*) como entre **SAs** (*rutado inter-autónomo*).

4.4.1 RIP (Routing Information Protocol):

Usa el rutado intra-autónomo, basado en **vector de distancias (VD)**. **Funciona sobre UDP** (puerto 520). En este protocolo, cada router **mantiene una tabla con las distancias a otros routers de la red**. **RIP** intercambia información solo entre routers vecinos, con el costo de cada enlace establecido en 1, y puede enviar paquetes a través de un **máximo de 15 saltos**. Si un router desea información sobre una ruta, envía un mensaje de petición a sus vecinos y recibe como respuesta una lista de redes destino con su distancia.

- **Mensajes de petición RIP:** solicitan información
- **Mensajes de respuesta RIP:** envían una lista de hasta 25 redes internas al SA (estos también se envían de forma automática cada 30. En caso de que no se reciba respuesta en 180 segundos, se considera caído).

4.4.2 OSPF (Open Shortest Path First):

Protocolo de enrutamiento intra-autónomo basado en el **estado de enlace (EE)** para intradominios. **Funciona sobre su propio protocolo de red**, utilizando el puerto 89.

OSPF es un protocolo **avanzado en comparación con RIP**, diseñado para reemplazarlo en redes de mayor complejidad. Utiliza un algoritmo basado en el estado de enlace, **difundiendo información sobre la red a todos los nodos** cuando ocurren cambios significativos, y actualizándose automáticamente cada 30 minutos. Cada router mantiene un mapa completo de la red, lo que permite **calcular el camino más corto mediante el algoritmo de Dijkstra**. Los mensajes **OSPF** incluyen mensajes **HELLO** enviados periódicamente a cada vecino para verificar su conectividad, y permite también consultar a un vecino para obtener la información de estado completa. Los costos de cada enlace son definidos por el administrador de la red.

OSPF es más seguro que RIP, ya que utiliza un **protocolo de transporte propio** en el que todos los mensajes están autenticados, permitiendo solo la comunicación entre routers autenticados. Siempre se elige el camino más corto disponible, y en caso de varios caminos con el mismo costo, OSPF puede repartir el tráfico entre ellos.

OSPF permite la subdivisión del Sistema Autónomo (SA) en áreas jerárquicas, lo que mejora la organización. Cada área ejecuta OSPF internamente, y las comunicaciones entre áreas se gestionan mediante routers de frontera de área (ABR), que encaminan el tráfico hacia fuera del área. Estos ABRs están conectados entre sí en una **área troncal**. Para enrutar fuera del SA, OSPF también utiliza un router de frontera del SA (ASBR).

4.4.3 BGP (Border Gateway Protocol):

Protocolo de enrutamiento inter-autónomo, **basado en vector de rutas**. **Funciona sobre TCP** (puerto 179).

BGP es el **protocolo estándar en Internet** para la **comunicación entre sistemas autónomos** (SAs) en su versión BGP4. Está diseñado para el **enrutamiento entre dominios y permite a los routers frontera** (routers pasarela) **intercambiar rutas completas** entre SAs. A diferencia de los protocolos de enrutamiento por distancia, BGP utiliza un vector de rutas, donde cada router comparte la ruta completa a cada destino, permitiendo la aplicación de políticas de enrutamiento específicas. En BGP, se usa **E-BGP** para la comunicación entre routers de diferentes SAs y **I-BGP** para la comunicación entre routers dentro del mismo SA.

Identificación de SAs: Cada SA en BGP se identifica mediante un número de sistema autónomo (ASN) único, facilitando el control y la organización de las rutas.

Ejemplo: Supongamos que un mensaje BGP describe cómo llegar a la red `128.19.4.0/24`. La ruta podría ser algo como `SA1/SA2/SA3/SA4`, lo que indica el camino exacto que el paquete debe seguir al atravesar distintos SAs.

Políticas de Enrutamiento: BGP permite a los administradores de cada SA definir políticas de enrutamiento específicas, como evitar redirigir el tráfico a través de ciertos proveedores o filtrar rutas hacia determinados destinos. Esto permite un control granular sobre el flujo de tráfico entre SAs.

4.4.4 Resumen - **RIP**: Protocolo de enrutamiento sencillo que intercambia solo la distancia a redes vecinas y está limitado a un máximo de 15 saltos. - **OSPF**: Protocolo de estado de enlace que utiliza un mapa completo de la red, actualizando la información periódicamente y calculando el camino más corto con el algoritmo de Dijkstra. Permite balanceo de carga entre rutas de igual costo y organización jerárquica mediante áreas. - **BGP**: Protocolo de enrutamiento inter-autónomo que intercambia rutas completas entre SAs, permitiendo la aplicación de políticas de enrutamiento detalladas y el control granular de rutas según las necesidades de cada operador.

Este sistema asegura que dentro de cada SA se determine la ruta óptima de forma eficiente, mientras que entre SAs los paquetes siguen rutas específicas optimizadas y controladas por cada operador en función de sus políticas de enrutamiento.

4.5 Protocolo de Internet (IP)

Define el formato de las **direcciones IP**, los campos de los **datagramas**, y **cómo los routers procesan estos datagramas**. Los routers leen estos campos para dirigir el tráfico a su destino correcto. Fue diseñado para conectar redes diferentes. La fiabilidad recae sobre las capas superiores (*TCP*).

4.5.1 Direccionamiento IPv4

Las direcciones **IPv4** son números de 4 bytes (32 bits) que se expresan en un formato de cuatro números decimales separados por puntos, donde cada número representa 1 byte (8 bits). Un ejemplo de dirección IPv4 es `192.168.1.1`. Cada dispositivo conectado a Internet (hosts, routers, etc.) tiene **una dirección IP por cada interfaz** (*punto de conexión a la red*). Por ejemplo, en Linux, una interfaz de tarjeta Ethernet puede llamarse `eth0`, una conexión serie `ppp0`, una conexión USB `usb0`, y una conexión inalámbrica `wlan0`.

Direcciones IP

Una dirección IP tiene dos partes:

- **Parte de red**: Identifica la red en la que está el dispositivo.
- **Parte de host**: Identifica el dispositivo específico en esa red.

Clases de Dirección IP: Antiguamente las direcciones IP se agrupaban en clases, según los primeros bits de la dirección:

- **Clase A**: Comienza con `0`. Tiene una gran cantidad de hosts por red, ideal para organizaciones grandes.
- **Clase B**: Comienza con `10`. Es adecuada para empresas grandes y medianas. Ejemplo: `172.16.0.0`.
- **Clase C**: Comienza con `110`. Se utiliza para pequeñas redes. Ejemplo: `192.168.0.0`.
- **Clase D**: Comienza con `1110`, destinada para **multicast** (envío de datos a varios dispositivos).
- **Clase E**: Comienza con `1111`, reservada para usos futuros.

Algunas direcciones reservadas por el IANA:

- **0.0.0.0** → Esta red. Usada para arrancar sistemas sin disco (como en el protocolo DHCP) y en el encaminamiento por defecto.
- **127.0.0.0 – 127.255.255.255** → La propia estación (*dirección de loopback*). Comúnmente se usa la **127.0.0.1**.
- **240.0.0.0 – 255.255.255.254** → Reservadas para uso futuro.
- **255.255.255.255** → Difusión a toda la red (usada en el protocolo DHCP).

INFO

En cada red, existe una **dirección de broadcast** que se utiliza para enviar mensajes a **todos los dispositivos de la red al mismo tiempo**. Por ejemplo, si un router necesita enviar una actualización de red o un dispositivo quiere buscar otros dispositivos conectados, puede enviar un mensaje a la dirección de broadcast y todos los equipos dentro de esa red lo recibirán. Para calcularla basta con poner todos los bits de la parte de host a 1. Son las direcciones que terminan por **255**. Las direcciones que acaban en **0** también son especiales porque **permiten identificar a la red**.

4.5.2 Subredes

Subredes son divisiones más pequeñas dentro de una red. Crear subredes permite dividir un bloque grande de direcciones IP en varias redes más pequeñas, cada una con su propio rango de direcciones. Las subredes son útiles para mejorar el rendimiento, la organización y la seguridad en una red. Para subdividir una red en varias subredes usamos **máscaras**.

Las **subredes** nos permiten **ahorrarnos IPs** porque las asignan de forma más eficiente, adaptándose a la cantidad de dispositivos en cada parte de la red. Además las redes grandes pueden **ser difíciles de gestionar** por lo que dividirla en subredes ayuda a segmentar el tráfico y tener mayor control. Al aislar cada subred se obtiene **mayor seguridad** limitando la comunicación entre dispositivos de diferentes subredes.

IMPORTANTE

La dirección base y la de broadcast están reservadas.

Máscara de subred

La máscara de subred se utiliza para **dividir una red IP en subredes más pequeñas**, usando parte del campo de la dirección de **red** para identificar la subred. Una máscara de subred es un **conjunto de 32 bits**, donde los primeros **n** bits (*los más significativos*) están en 1 para indicar la porción de red y subred, y los restantes **32 – n** bits están en 0 para **identificar los hosts** dentro de esa subred.

- **Formato:** Las máscaras se representan en notación de longitud de prefijo (por ejemplo, `/27`) o en formato decimal (por ejemplo, `255.255.255.224`).
- **Ejemplo de máscara /27:**
 - `/27` significa que los primeros 27 bits están en 1 y los últimos 5 en 0.
 - En formato binario: `11111111.11111111.11111111.11100000`, equivalente en decimal a `255.255.255.224`.

Consideremos una `192.168.17.0` con una máscara de subred `/27` (o `255.255.255.224`):

Los primeros 24 bits (`192.168.17`) representan la red principal. Los siguientes 3 bits indican la subred específica dentro de esta red. Los últimos 5 bits identifican el host o estación dentro de esa subred.

Número de subredes: Como tenemos 3 bits para la subred, se pueden crear $2^3 = 8$ subredes.

Número de hosts por subred: Con 5 bits para hosts, cada subred puede tener $2^5 - 2 = 30$ estaciones (se resta 2 porque no se pueden usar las direcciones de subred y de difusión). **Total de direcciones posibles:** $8 \text{ subredes} \times 30 \text{ hosts} = 240 \text{ direcciones}$.

Redes sin clase, direcciones CIDR (Classless Inter-Domain Routing)

En 1993, el sistema tradicional de direcciones IP basado en clases (A, B, C) fue reemplazado por **CIDR (Classless Inter-Domain Routing)**. Este cambio permitió dividir las direcciones IP en redes y hosts con mayor flexibilidad, lo cual ayudó a mejorar la administración de redes y el uso eficiente de las direcciones IP.

En CIDR, una dirección IP **no está restringida a una clase fija**. En su lugar, la división entre la parte de **red** y la parte de **host** de la dirección IP puede ubicarse en cualquier posición. Esto se indica mediante la **máscara** en notación `/x`, donde `x` es el número de bits dedicados a la red.

Dirección CIDR: `193.144.48.0/20` Esto indica que los primeros 20 bits representan la **red**, y el resto de los bits, los **hosts**.

En binario: `1100 0001 1001 0000 0011 0000 0000 0000`

- **Parte de red:** Primeros 20 bits → `193.144.48.0`
- **Parte de host:** Últimos 12 bits → identifican dispositivos dentro de la red `193.144.48.0/20`

Diferencia Redes con Clase y CIDR

La diferencia entre el esquema tradicional de clases de red y las redes sin clase (o CIDR, **Classless Inter-Domain Routing**) radica en cómo se asignan y manejan los rangos de direcciones IP y la flexibilidad para dividir y agrupar las redes.

Redes con Clase (Clases A, B, C)

En el esquema de redes con clase: Las direcciones IP se dividen en clases (A, B, C, D y E), y cada clase tiene un rango de direcciones IP fijo y una longitud de prefijo predeterminada:

- **Clase A:** Prefijo /8 (los primeros 8 bits identifican la red), cubriendo un rango de IPs muy amplio.
- **Clase B:** Prefijo /16 (los primeros 16 bits identifican la red).
- **Clase C:** Prefijo /24 (los primeros 24 bits identifican la red).

La máscara de subred era implícita para cada clase y no se podía cambiar. Esto significa que una red de clase C, por ejemplo, siempre tendría una máscara de `255.255.255.0` (/24), lo que permite un máximo de 254 hosts por red. El esquema es **rígido** y puede llevar a desperdicio de direcciones IP. Por ejemplo, si una organización necesitaba más de 254 hosts, no podía usar clase C y tendría que usar una clase B, que proporcionaba 65,534 direcciones, mucho más de lo necesario.

Redes Sin Clase (CIDR)

CIDR fue introducido para superar las limitaciones del esquema de clases y permitir una asignación más flexible y eficiente de las direcciones IP.

En CIDR, **no se utilizan las clases tradicionales** (A, B, C) para determinar el tamaño de la red. En cambio, se usa una notación de prefijo (por ejemplo, `/18`, `/24`, `/12`, etc.), que indica cuántos bits de la dirección IP identifican la red.

Con CIDR, puedes tener una **máscara de subred de cualquier longitud**, independientemente de la "clase" de la dirección IP. Esto permite subdividir redes grandes o combinar redes pequeñas sin las restricciones de las clases: Por ejemplo, `192.168.1.0/26` define una red que usa 26 bits para la porción de red y 6 bits para hosts, lo que permite tener 4 subredes dentro de lo que tradicionalmente sería una red de clase C.

CIDR permite crear subredes más pequeñas o "superredes" para asignar solo el número exacto de direcciones IP necesario, reduciendo el desperdicio de direcciones. También permite la **agregación de rutas** en la tabla de enrutamiento, simplificando la gestión de redes en Internet.

Tablas de Reenvío

Tabla de Reenvío de un Host

La tabla de reenvío es la estructura que un host utiliza para determinar cómo enviar paquetes a su destino. Ejemplo para el host `200.1.3.3`:

Red destino	Gateway	Interfaz
<code>127.0.0.0/8</code>	*	<code>lo</code>
<code>200.1.3.0/24</code>	*	<code>eth0</code>
<code>0.0.0.0/0</code> (por defecto)	<code>200.1.3.1</code>	<code>eth0</code>

- **127.0.0.0/8** → Dirección de **loopback** (interfaz `lo`), utilizada para **pruebas locales** en el host.
- **200.1.3.0/24** → Red local, por lo que los paquetes pueden salir directamente por la interfaz `eth0`.

- **0.0.0.0/0** (por defecto) → Cualquier otra dirección de destino se envía a través de **200.1.3.1** usando **eth0**.

Para consultar la tabla de rutas en Linux, usa el comando:

```
route [-n]
```

Tabla de Reenvío de un Router

La tabla de reenvío en un router incluye información detallada sobre las redes conectadas, interfaces y gateways.

Destino	Interfaz	Gateway	Métrica
194.24.0.0/21	inti	*	i saltos
194.24.8.0/22	intj	*	j saltos
194.24.16.0/20	intk	*	k saltos
0.0.0.0/0 (por defecto)	intx	*	x saltos

[! Info] El **gateway** (o puerta de enlace) es la dirección de un dispositivo de red que sirve de punto de acceso para enviar datos a otras redes. En una tabla de reenvío de un router, el gateway indica hacia dónde debe dirigirse el tráfico que no pertenece a la red local. Es especialmente útil cuando el tráfico necesita salir de la red local hacia redes más amplias, como internet o redes externas. El gateway está marcado con *****, lo cual indica que para las rutas especificadas no es necesario un gateway intermedio, ya que el tráfico puede enviarse directamente a través de la interfaz correspondiente. Por ejemplo el router de casa.

Reenvío de un paquete: coincidencia del prefijo más largo

Agregación de rutas

Es un proceso que realizan los routers por el cual toman un **grupo de direcciones de redes contiguas** (bloque CIDR) y las resumen en una sola dirección de red común a todas esas redes.

La ventaja principal es la **optimización del enrutamiento** en grandes redes corporativas, ya que los routers tienen que **mantener menos entradas en sus tablas de enrutamiento** y en consecuencia ganan estabilidad. Si un router tiene conectadas 10 redes contiguas, solo publicará el resumen de la ruta CIDR a sus vecinos.

Para obtener la dirección IP **base de agregación de rutas** de debemos comprobar que sean redes contiguas y, a continuación, **hacer la operación AND sobre las direcciones base de esas redes**. Para obtener la dirección de broadcast debemos poner todos los bits que tenían comunes al hacer la operación AND a 1.

En la imagen podemos ver que en la tabla de reenvío del **router B** tendríamos una entrada para A, B, C, D y E, lo que pasa es que C, D y E van a ser consecutivas y reenviarse por la misma interfaz, así que se agrupan en **una única entrada** y cuando llegue un paquete de cualquiera de esas redes van a ser reenviadas por al **interfaz RB2** y cuando llegue al **router C** ya se decidirá a donde va cada paquete.

4.5.3 Datagrama IP

NOTA

Saber diferencias datagrama IPv4 e IPv6

Versión (4 bits): Indica la versión del protocolo IP utilizado. Normalmente es IPv4 (valor 4) o IPv6 (valor 6). **Longitud de Cabecera (lg. cab.) (4 bits):** Muestra la longitud de la cabecera en múltiplos de 4 bytes. Esto permite determinar dónde comienza la sección de datos. **Tipo de Servicio (8 bits):** Define la prioridad y calidad del servicio del paquete, permitiendo especificar si debe ser tratado con baja latencia, alta fiabilidad, etc. **Longitud Total (16 bits):** Es la longitud total del paquete IP, incluida la cabecera y los datos. Permite al receptor saber cuándo termina el paquete. **Identificador (16 bits):** Usado para identificar de forma única un fragmento de un datagrama IP. Es útil cuando un paquete debe dividirse en fragmentos para atravesar una red con un tamaño de unidad de transmisión más pequeño. **Indicadores (indicad.) (3 bits):** Indican si el paquete puede ser fragmentado y si es el último fragmento de un conjunto de fragmentos.

Desplazamiento de Fragmento (despl. fragmento) (13 bits): Indica la posición de un fragmento dentro del datagrama original, facilitando la reconstrucción del paquete en el receptor.

$$\text{Desplazamiento (en unidades de 8 bytes)} = \frac{\text{Posición del inicio del fragmento (en bytes)}}{8}$$

Tiempo de Vida (8 bits): Controla el tiempo de vida del paquete, especificando el **número máximo de saltos** (routers) que puede atravesar. Se decrementa en cada router, y cuando llega a cero, el paquete es descartado.

Protocolo (8 bits): Indica el protocolo de la capa superior que se encuentra en los datos del paquete (por ejemplo, TCP o UDP). Esto ayuda a dirigir el paquete al protocolo adecuado. **Suma de Comprobación de Cabecera (16 bits):** Campo de verificación para la cabecera IP. Permite comprobar la integridad de la cabecera, detectando errores en la transmisión. **Dirección IP Origen (32 bits):** La dirección IP del dispositivo que envió el paquete. Permite al receptor saber de dónde proviene el paquete. **Dirección IP Destino (32 bits):** La dirección IP del destinatario previsto del paquete, necesaria para la entrega del mismo. **Opciones (variable):** Este campo es opcional y se utiliza para habilitar características avanzadas, como pruebas de conectividad y enrutamiento específico. Si no se utiliza, la longitud del campo es cero. **Datos (variable):** Contiene la carga útil del paquete IP, que puede ser información de una aplicación o de un protocolo de nivel superior, como TCP o UDP.

4.5.4 Fragmentación de Datagramas

La fragmentación de datagramas es un proceso que permite que los datagramas IP puedan atravesar diferentes tipos de redes, cada una con su propio límite de tamaño de paquete permitido, conocido como **MTU** (*Maximum Transmission Unit*). Por ejemplo, Ethernet tiene un **MTU máximo de 1500 bytes**, mientras que algunas redes de área extensa solo admiten paquetes de 576 bytes. Cuando un datagrama excede el tamaño permitido por la red, **se fragmenta en partes más pequeñas**.

Los routers son responsables de fragmentar los datagramas cuando es necesario, según las restricciones de **MTU** de la red. Los fragmentos no se reensamblan durante el trayecto, sino que se espera hasta que los fragmentos lleguen al destino final. En ese momento, la **capa de red del host receptor se encarga de reensamblar los fragmentos** y entregarlos a la capa de transporte. Los **host** también pueden fragmentar, pensad en el ejemplo de conectar a un ordenador con un router y que el ordenador le manda datos al router y el cable usado es una mierda, debe segmentar IP en el host y no solo en router.

Para minimizar la **fragmentación**, los protocolos TCP y UDP tienden a generar segmentos de **tamaño más pequeño**. Esto permite que las cabeceras de IP y TCP (alrededor de 20 bytes cada una) se incluyan dentro de los límites de tamaño de las redes comunes, evitando la fragmentación innecesaria.

NOTA

TCP cuando detecta que se sobrepasa el **MSS** se lo pasa a IP por lo que ya segmenta antes, aunque si el **MSS** más la cabecera IP es mayor que el **MTU** de alguna de las redes por las que el router va a reenviar el paquete, lo segmenta aún más. **UDP** directamente no lo segmenta y se pasa del **MTU** de la red por la que se va a reenviar se segmenta en el router.

Fragmentar->capa de red Segmentar->capa de transporte

En cada red existe un valor de MTU que se aplica a los datagramas que se envían. Si un datagrama excede el MTU de la red, este se fragmenta en unidades más pequeñas. A cada fragmento se le añaden **cabeceras de IP**. Durante la fragmentación, se utiliza el identificador **MF** (*More Fragments*) en cada fragmento para indicar que quedan más partes de ese datagrama, a excepción del último fragmento, cuyo bit **MF** se establece en **0**. Además, si el bit **DF** (*Don't Fragment*) está activado en el datagrama original, este no será fragmentado y **será descartado si no cabe en la red**.

IMPORTANTE: Desplazamiento (en unidades de 8 bytes) = $\frac{\text{Posición del inicio del fragmento (en bytes)}}{8}$

Este proceso de fragmentación y reensamblaje garantiza que los datagramas puedan adaptarse a diferentes redes con diferentes capacidades de transmisión, permitiendo la comunicación entre sistemas que podrían estar conectados por redes con MTU variables.

4.5.5 Seguridad en IPv4

El rastreo de paquetes es una técnica que permite a un atacante interceptar y **analizar los paquetes que se transmiten a través de una red**. Este ataque es pasivo, ya que el atacante **no modifica el contenido de los paquetes**, solo los observa y captura.

- **Solución:** El **cifrado** de los datos garantiza que, aunque un atacante logre capturar los paquetes, no podrá leer su contenido. Usando protocolos como **TLS** o **IPsec**, los datos se cifran y solo las partes autorizadas pueden acceder a la información original.

La modificación de paquetes, en este ataque, un atacante altera el contenido de los paquetes que se transmiten en la red. El receptor del **paquete modificado cree que el paquete proviene del remitente legítimo**, lo que puede dar lugar a acciones no deseadas o maliciosas.

- **Solución:** Los **mecanismos de integridad de datos** como **HMAC** o **sumas de comprobación** aseguran que los paquetes no se hayan modificado durante el tránsito. El receptor puede verificar que el contenido del paquete es auténtico y no ha sido alterado.

El **spoofing** es un ataque en el que un atacante **falsifica la dirección IP de origen de un paquete**, haciéndolo parecer como si proviniera de una fuente legítima. Esto puede permitir al atacante evadir medidas de seguridad o realizar ataques como la **denegación de servicio (DoS)**.

- **Solución:** La implementación de **mecanismos de autenticación del origen**, como **autenticación mutua** en **IPsec** o protocolos de validación de IP, garantiza que las direcciones de origen de los paquetes sean verificadas y auténticas, evitando suplantaciones.

IPSec

IPSec (Internet Protocol Security) es **un conjunto de protocolos de seguridad** utilizado para proteger las comunicaciones de red a nivel de IP. Se usa comúnmente para proporcionar seguridad en **redes privadas virtuales (VPNs)** y otras comunicaciones de red. IPSec puede ser usado en conjunto con IP para crear una red segura entre dos entidades. IPSec proporciona un servicio de seguridad que permite **establecer una conexión segura entre dos entidades**, lo que garantiza que los datos enviados a través de la red estén protegidos.

Servicios Proporcionados por IPSec:

Definición de Algoritmos y Claves: Las entidades que se comunican acuerdan los algoritmos de cifrado y las claves necesarias para asegurar la comunicación. Esto puede incluir algoritmos como **AES** para cifrado y **SHA** para la integridad de los datos.

Cifrado de Paquetes: IPSec cifra los datos antes de enviarlos a través de la red, asegurando que, aunque los datos sean interceptados, no puedan ser leídos sin las claves adecuadas.

Integridad de los Datos: Para evitar la modificación de los paquetes, IPSec implementa mecanismos de verificación de integridad. Esto asegura que los datos no han sido alterados durante la transmisión.

Autenticación del Origen: IPSec también proporciona mecanismos de autenticación que verifican que los paquetes provienen de la fuente legítima y no de un atacante que esté suplantando la identidad del remitente.

4.5.6 Direccionamiento IPv6

IPv6 es la versión más reciente del protocolo de Internet que reemplaza al antiguo IPv4. La transición de IPv4 a IPv6 responde principalmente al **agotamiento de direcciones IP disponibles en IPv4** (debido a su límite de 32 bits) y a la necesidad de un sistema más eficiente y escalable para manejar el creciente número de dispositivos conectados a Internet.

En IPv4, las direcciones IP son de 32 bits, lo que da como resultado alrededor de **4.3 mil millones de direcciones posibles**. Sin embargo, con el rápido crecimiento de dispositivos conectados, estas direcciones se agotaron. En cambio, **IPv6 utiliza direcciones de 128 bits**, lo que da como resultado una cantidad prácticamente ilimitada de direcciones (aproximadamente 340 sextillones de direcciones).

Las direcciones IPv6 tienen 8 bloques de 16 bits en hexadecimal, separados por dos puntos. Ejemplo:

```
47CD:1234:4422:AC02:0022:1234:A456:0124
```

Los bloques consecutivos de ceros se representan como `::` una sola vez por dirección:

```
47CD:0000:0000:0000:0000:0000:A456:0124 ⇒ 47CD::A456:0124
```

Las direcciones IPv4 se representan en IPv6 para facilitar la transición, con `::FFFF:` seguido de la dirección IPv4. Ejemplo:

```
::FFFF:193.144.84.77
```

En IPv6, las máscaras indican la parte de la dirección que define la red. Ejemplo:

- Dirección: `3ffe:ffff:100:1:2:3:4:5/48`
- Red derivada con máscara `/48`: `3ffe:ffff:0100:0000:0000:0000:0000:0000`

Tipos de direcciones de IPv6:

- **Unicast:** Identifica un único host.
- **Multicast:** Identifica un grupo de hosts (empiezan con `FF`).
- **Anycast:** Envía el paquete al host más cercano en la red.

IPv6 mejora la flexibilidad y eficiencia en comparación con IPv4.

4.5.7 Formato del datagrama IPv6

Versión (4 bits): Indica la versión del protocolo IP. Para IPv6, este valor es 6. **Clase de Tráfico (8 bits):** Define la prioridad y clase de tráfico del paquete, permitiendo especificar si debe ser tratado con baja latencia, alta prioridad, etc. **Etiqueta de Flujo (Flow Label) (20 bits):** Utilizado para identificar flujos de paquetes que requieren un manejo especial (como los de una misma conexión o flujo de datos). Ayuda a los routers a procesar los paquetes más eficientemente. **Longitud de Datos (Payload Length) (16 bits):** Especifica la longitud de los datos contenidos en el paquete, excluyendo la cabecera. **Siguiente Cabecera (Next Header) (8 bits):** Indica el tipo de cabecera que sigue a esta cabecera IPv6. Puede ser un protocolo de capa superior (como TCP o UDP) o una cabecera de extensión de IPv6. **Límite de Saltos (Hop Limit) (8 bits):** Similar al "Tiempo de Vida" (TTL) en IPv4, especifica el número máximo de saltos que el paquete puede realizar antes de ser descartado. Se decrementa en cada router que atraviesa. **Dirección IPv6 Origen (128 bits):** Contiene la dirección IP del dispositivo que envió el paquete, permitiendo que el destinatario sepa de dónde proviene. **Dirección IPv6 Destino (128 bits):** Contiene la dirección IP del dispositivo de destino, necesaria para la entrega del paquete. **Datos (variable):** Contiene la carga útil del paquete, que incluye la información de la aplicación o de un protocolo de nivel superior.

La cabecera IPv6 tiene un **tamaño fijo de 40 bytes y es más simple que la de IPv4**, eliminando algunos campos y facilitando el procesamiento. Esta estructura fue diseñada para mejorar la eficiencia y el enrutamiento en redes IPv6.

En IPv6, se ha eliminado la **fragmentación en los routers**. En IPv4, los routers fragmentan los paquetes si son demasiado grandes para la red siguiente. En IPv6, los routers **no fragmentan los paquetes**, y si un paquete es demasiado grande, se descarta y se envía un mensaje **ICMP** ("paquete demasiado grande") al emisor, quien debe reenviar el paquete en fragmentos más pequeños. La **fragmentación y reensamblado** se realiza **únicamente en el host de origen** y no en los routers intermedios, lo que mejora la eficiencia en el procesamiento de paquetes.

Se elimina también la **suma de comprobación** dejándole a las capas superiores la tarea de comprobarlo. Y los paquetes **ya no se numeran**.

4.5.8 Transición de IPv4 a IPv6

La transición de IPv4 a IPv6 es un proceso gradual en el cual ambos protocolos coexistirán durante un tiempo. Una de las técnicas más comunes para facilitar esta transición es el **tunneling**. Esta técnica permite que los paquetes IPv6 puedan atravesar redes que solo soportan IPv4 mediante la encapsulación.

Consiste en **encapsular** los paquetes IPv6 dentro de paquetes IPv4. Para ello **se añaden cabeceras IPv4** con las direcciones de los routers que no soportan IPv6. Cuando llegan a zonas de **IPv6** se **eliminan dichas cabeceras**.

4.6 VPN (Redes Privadas Virtuales)

Es una red utilizada por una organización que utiliza la red **pública** para comunicarse de forma segura, en vez de tener que crear una propia red, lo que es muy costoso.

Se usan sistemas de **encriptación** y **autenticación**.

- **IPSec**: protocolo de seguridad de IP para soportar comunicaciones seguras e implementar VPNs
- También se pueden implementar mediante **túneles IP**

Túneles IP - El ****paquete original se encapsula dentro de un nuevo paquete IP**** que puede ser transportado por la red subyacente. - El ****paquete encapsulado se envía a través de una red intermedia****, que ****no entiende el contenido, pero si la cabecera que lo encapsula****. - En el extremo del túnel, ****el encabezado se elimina****, y el paquete original se entrega al destino final.

4.7 ICMP (Internet Control Message Protocol)

ICMP es un protocolo de control que **funciona sobre IP** y permite a **hosts y routers informar sobre errores o problemas de la red**. Aunque opera en la capa de red, no transporta datos de usuario, sino que envía mensajes para mantener el estado y la integridad de la red. **Su entrega no está garantizada**, ya que se encapsula en un datagrama IP.

Cada mensaje ICMP incluye:

- **Tipo y Código**: Definen la naturaleza del mensaje (por ejemplo, solicitud de eco, destino inaccesible, tiempo excedido).
- **8 primeros bytes del datagrama original**: Ayudan a identificar el paquete que causó el error.

Destino inalcanzable

Lo envía un nodo a la estación origen cuando no puede alcanzar el destino o cuando el datagrama no puede fragmentarse y no puede atravesar una red (tipo ICMP 3).

Tiempo excedido

Cuando un nodo destruye un datagrama porque su contador llegó a 0 lo manda a la estación origen (tipo ICMP 11).

Ralentizar fuente

Para limitar el número de datagramas que las estaciones introducen en la red y evitar la congestión (tipo ICMP 4).

Solicitud de eco y Respuesta de eco

Se utilizan para ver si un destino es alcanzable y se encuentra activo (uso en ping) (tipos ICMP 8 y 0).

Problema de parámetro

Indica que se ha detectado un valor ilegal en un campo de la cabecera (tipo ICMP 12).

Redirigir

Se utiliza cuando un nodo se da cuenta de que hay un mejor camino para enviar el datagrama.

Marca de tiempo y Respuesta a marca de tiempo

Para medir el retardo de la red

Petición de máscara de dirección y Respuesta de máscara de dirección

Empleadas cuando se usan subredes, permiten a un computador conocer la máscara de subred

4.8 DHCP (Dynamic Host Configuration Protocol)

DHCP es un protocolo que **permite asignar direcciones IP a los hosts de forma automática y dinámica**, ideal para redes donde **no hay suficientes IPs para todos los dispositivos**, como en redes de ISPs o redes inalámbricas.

Estáticamente: las direcciones las asigna el administrador del equipo. **Dinámicamente:** con DHCP.

Cuando un dispositivo **se conecta a la red, el proceso DHCP se realiza en cuatro pasos:**

1. **Descubrimiento (DHCPDISCOVER):** El host envía un mensaje **broadcast** con una IP de origen en ceros y la IP de destino en 255.255.255.255, **buscando un servidor DHCP disponible**.
2. **Oferta (DHCPOFFER):** El servidor DHCP responde con **una dirección IP, su máscara de red, el tiempo de concesión de la IP, el gateway predeterminado y los servidores DNS disponibles**.
3. **Solicitud (DHCPREQUEST):** El cliente **selecciona una de las ofertas** (*si recibió varias*) y responde indicando la IP que desea aceptar.
4. **Confirmación (DHCPACK):** El servidor DHCP envía una confirmación (ACK) al cliente, estableciendo la asignación de la IP para el tiempo acordado.

4.9 NAT (Network Address Translation)

NAT es un sistema que permite que varios dispositivos dentro de una red privada **usen una misma IP pública válida para conectarse al exterior**. Utiliza direcciones especiales para redes privadas, que los routers de Internet ignoran. Estas son:

- **Clase A:** 10.0.0.0/8
- **Clase B:** 172.16.0.0/12
- **Clase C:** 192.168.0.0/16

Un servidor NAT tiene dos interfaces con direcciones IP distintas:

- **IP pública:** Para conectar al exterior.
- **IP privada:** Para conectar a la red interna.

Los dispositivos de la red privada configuran la IP privada del NAT como su **gateway**, y este modifica los **datos de origen** (IP y puerto) de cada paquete antes de enviarlo al exterior. También mantiene una **tabla de redirección** para asociar cada paquete de respuesta con el dispositivo interno correspondiente.

5. Capa de Enlace

Escrito por **Adrián Quiroga Linares**.

5.1 Introducción

La **capa de enlace** es la segunda capa del modelo OSI y se encarga de gestionar la transmisión de datos entre nodos directamente conectados en una red. Su propósito principal es asegurar que los datos se transfieran de forma confiable a través de un enlace físico entre los extremos de una conexión.

- **Nodos:** hosts origen y destino y los routers
- **Enlaces:** LANs o redes punto a punto

El objetivo principal de esta capa es transmitir bloques de bits de un lado a otro. La capa de enlace organiza los datos en unidades manejables llamadas *tramas* (*frames* o *marcos*), las cuales encapsulan información del nivel superior junto con metadatos para su correcta transmisión y recepción.

La unidad de medida o *PDU* (*Protocol Data Unit*) de la capa de enlace es la **trama**. Esta incluye:

- **Cabecera (*header*):** contiene información de control, como direcciones físicas y mecanismos de detección de errores.
- **Datos (*payload*):** la información útil.
- **Cola (*trailer*):** bits adicionales usados para la detección y corrección de errores.

La capa de enlace se implementa generalmente en la **tarjeta de red** del dispositivo. El sistema operativo transfiere los datos al adaptador de red. Este adaptador añade la cabecera y envía las tramas al medio físico.

Encontramos **dos tipos de enlaces**:

1. **Punto a Punto:** Enlace directo entre un emisor y un receptor (un único par de dispositivos).
2. **Difusión:** Un medio compartido donde múltiples emisores y receptores pueden comunicarse (ejemplo: redes Ethernet).

Los protocolos en esta capa definen el **formato de las tramas** y las acciones a realizar cuando un nodo envía o recibe tramas. La **capa de red** encapsula los datos de las capas superiores en tramas, delimitando dónde empieza y termina cada trama. **Controla el acceso al medio físico** mediante protocolos de acceso múltiple (*MAC*), esenciales en medios compartidos. Asegura que las tramas lleguen al receptor mediante confirmaciones y retransmisiones si es necesario. **Regula la velocidad de envío** para evitar saturar al receptor. Utiliza técnicas avanzadas para **identificar errores** en los datos causados por interferencias electromagnéticas o ruido en la señal. **Repara errores** detectados

mediante mecanismos integrados en hardware o software. **Especifica el modo de Transmisión:** - **Half-Duplex:** Transmisión en un solo sentido a la vez. - **Full-Duplex:** Transmisión simultánea en ambos sentidos.

RECORDATORIO

5.2 Modelo IEEE 802

El **modelo IEEE 802** define estándares para **redes de área local (LANs)** y **redes de área metropolitana (MANs)**. Este modelo organiza la **capa de enlace** en dos subcapas principales para manejar la comunicación y garantizar la interoperabilidad entre dispositivos de diferentes fabricantes.

5.2.1 LLC (Control de Enlace Lógico) La **capa LLC** actúa como una interfaz entre las capas superiores (como la capa de red) y la subcapa MAC. Se encarga del **control de errores y de flujo**.

Sin conexión ni confirmaciones:

- No verifica que los datos lleguen correctamente.
- No hay control de flujo ni control de errores.
- Las capas superiores son responsables de gestionar problemas como retransmisiones. **Sin conexión con confirmaciones:**

- Confirma la recepción de las tramas individuales.
- No establece una conexión previa (es decir, no asegura que haya una sesión de comunicación predefinida). **Con conexión y confirmaciones:**
- Establece una conexión lógica antes de la transmisión.
- Asegura el control de flujo y la detección/corrección de errores.

5.2.2 MAC (Control de Acceso al Medio)

La **capa MAC** es responsable de **ensamblar** los datos en *tramas* (frames) y **gestionar el acceso al medio físico compartido**. Especifica cómo los dispositivos en una red **comparten el medio de transmisión** y garantiza que las tramas lleguen al destino correcto.

Maneja la transmisión en medios compartidos, controlando quién puede enviar en cada momento (evita colisiones). **Detecta y corrige errores** en las tramas cuando sea posible. **Desensambla** las tramas en el receptor, verificando las direcciones y detectando errores.

5.2.3 Relación Entre LLC y MAC

La subcapa **LLC** es **independiente de la tecnología del medio físico**. Esto significa que los **mismos mecanismos de control de flujo y errores pueden usarse sobre cualquier tecnología subyacente**. La subcapa **MAC**, en cambio, está **directamente relacionada con el medio de transmisión** y varía según el tipo de red: - Ethernet tiene sus propias reglas para el acceso al medio, diferentes de las usadas en Token Ring o Wi-Fi. **Un LLC puede trabajar con múltiples MACs**, lo que hace al modelo IEEE 802 modular y flexible.

5.3 Direcciones MAC Ethernet

La arquitectura de redes combina diferentes tipos de direcciones para identificar dispositivos y facilitar la comunicación. En este contexto, las **direcciones MAC** y las **direcciones IP** desempeñan roles complementarios. La **dirección MAC (Media Access Control)** es un **identificador único asignado a cada adaptador de red o interfaz de red**. Es una dirección física, grabada generalmente en la memoria ROM del adaptador Ethernet durante su fabricación.

Tiene una longitud de **6 bytes** expresados en hexadecimal. Ejemplo: `00:1A:2B:3C:4D:5E`. Para garantizar que las direcciones no se repitan:

- Los primeros 3 bytes (24 bits) representan un **OUI (Organizationally Unique Identifier)**, asignado al fabricante por la IEEE.
- Los últimos 3 bytes (24 bits) son asignados por el fabricante para identificar de forma única cada dispositivo.

Se puede pensar que la **IP** indica la dirección física de un dispositivo (*la cual puede cambiar*) y la **MAC** es como su DNI (*no cambia*). **Direcciones especiales:**

- **Unicast:** Se usa para identificar un único dispositivo en la red. Una trama con una dirección unicast de destino es **procesada solo por el dispositivo cuyo adaptador tiene esa dirección MAC**.
- **Broadcast:** Direcciones con **todos los bits en 1** (`FF:FF:FF:FF:FF:FF`). Se utilizan para enviar tramas a **todos los dispositivos** de una red local.
- **Multicast:** Direcciones en las que el **bit menos significativo del primer byte** es `1`. Permite enviar tramas a un **grupo específico de dispositivos** en la red.
- **Modo promiscuo:** En este modo, un adaptador puede aceptar **todas las tramas**, independientemente de su dirección de destino. Útil para herramientas de diagnóstico y análisis de redes, como los analizadores de paquetes (sniffers). Puede activarse manualmente con comandos como `ifconfig eth0 promisc`.

> [!Importante] > ****Podemos ver que la IP no cambia a lo largo del recorrido, pero la MAC address sí.****

5.3.1 Protocolo ARP (Address Resolution Protocol)

El protocolo **ARP** es esencial para traducir las direcciones IP, que son lógicas y funcionan en la capa de red, a direcciones MAC, necesarias en la capa de enlace para la transmisión física de datos.

Cuando un dispositivo necesita enviar datos a una dirección IP, primero consulta la **tabla de caché ARP** para verificar si ya tiene la dirección MAC correspondiente. Si la entrada existe, se utiliza directamente. **Si la dirección IP no está en la tabla**, se envía una **solicitud ARP** a través de la red local usando una **trama de broadcast**. Ejemplo: "¿Quién tiene la dirección IP 192.168.1.10?". ****Después** viene la respuesta ARP donde el dispositivo con la dirección IP indicada responde con su dirección MAC en una **trama**. Ejemplo: "192.168.1.10 está en la dirección MAC 00:1A:2B:3C:4D:5E". **Finalmente actualiza la tabla**, la dirección MAC recibida se almacena en la **tabla de caché ARP** para uso futuro. Las entradas en la caché tienen una **duración limitada** (por defecto, 15 minutos).

5.4 Ethernet Ethernet es un estándar ampliamente utilizado para redes LAN (Local Area Network). Su simplicidad y versatilidad lo han convertido en el tipo de red más común. Se caracteriza por ser una red ****de difusión**** y proporcionar un servicio ****no fiable****, lo que significa que no garantiza la entrega de datos, dejando el control de errores a capas superiores (como TCP). ## 5.4.1 Formato de la trama Ethernet

Cabecera tiene **7 bytes** con el patrón 10101010 repetido. Permite sincronizar al receptor con el emisor. Además tiene el **SFD (Start of Frame Delimiter) 1 byte** con el patrón 10101011. Indica el inicio de la trama.

Longitud del campo de datos (2 bytes) (En Ethernet DIX, campo de Tipo, indica el protocolo de red usado (IP o ARP)) **Relleno**: para que la trama tenga un **tamaño mínimo**. Tamaño mínimo (sin cabecera ni SFD): **64 bytes** = 512 bits Tamaño máximo: **1518 bytes** = 12144 bits. **FCS** (Frame Check Sequence): código CRC de 4 bytes

5.4.2 Difusión y control de acceso: CSMA/CD

Ethernet utiliza el protocolo **CSMA/CD** (Carrier Sense Multiple Access with Collision Detection), un método para gestionar el acceso al medio compartido.

Todos los adaptadores conectados a la red están **constantemente escuchando el medio**. Antes de transmitir, el adaptador **verifica si el medio está ocupado**. Si el medio está **libre**, comienza a transmitir y si el medio está **ocupado**, espera a que quede libre y **deja un intervalo de seguridad** antes de intentar transmitir.

Se pueden producir **colisiones en el medio**. Una **colisión** ocurre cuando dos nodos transmiten al mismo tiempo y sus señales se superponen en el medio. Ethernet, al ser una red de difusión, es susceptible a estas colisiones.

El nodo transmisor escucha el medio mientras transmite. **Si detecta una colisión** (señal alterada), **detiene la transmisión**. Existe un período en el que un nodo puede creer que el medio está libre mientras otra señal está en tránsito. Para evitar esto:

NOTA

Tiempo mínimo de la trama = Tiempo de transmisión de la trama Así nos aseguramos de que tarde más en escribir la trama en el enlace que lo que tarda en propagarse la info 2 veces.

$$\text{Tiempo mínimo de la trama} > 2 \cdot \text{Tiempo de propagación (tamprop)}$$

Cuando se detecta una **colisión** termina de transmitir la cabecera de la trama. Al detectar una colisión, el nodo transmite una secuencia de **32 bits** conocida como **secuencia de interferencia** (*jamming sequence*), para garantizar que todos los nodos detecten la colisión. Detiene la transmisión y usa el algoritmo de **espera exponencial binaria**:

Divide el tiempo en ranuras de longitud proporcional a $T = 2 \cdot t_{\text{propMáx}}$ **Primera colisión:**

- El nodo espera un tiempo aleatorio de 0 o T antes de reintentar. **Colisiones subsecuentes: **
- El tiempo de espera aumenta exponencialmente:
 - Segunda colisión: 0, T , $2T$, $3T$.
 - Hasta 10 colisiones: 0 hasta $1023T$.

NOTA

El intervalo es $[0, (2^n - 1) * T]$

Límite de reintentos:

- Tras **16 colisiones consecutivas**, el nodo desiste e informa del fallo. Y las capas superiores se encargan de recuperar el fallo.

5.4.3 Ventajas y desventajas de Ethernet

Ventajas	Desventajas
Es sencillo y económico de implementar.	No garantiza la entrega de datos (no fiable).
Ofrece alta velocidad y escalabilidad.	Sensible a colisiones en entornos congestionados.
Es compatible con múltiples medios físicos.	El rendimiento decrece con un gran número de nodos.

Ethernet ha evolucionado desde sus primeras versiones para soportar redes más rápidas, confiables y eficientes, adaptándose a entornos modernos gracias a tecnologías como Ethernet full-duplex y switches que eliminan las colisiones.

5.5 Tecnologías Ethernet

5.5.1 Repetidores

Los repetidores son dispositivos de la **capa física** del modelo OSI que trabajan exclusivamente a nivel de **bits individuales**. Su función principal es **extender la distancia de transmisión copiando y regenerando los bits** que reciben por una interfaz y enviándolos al resto de interfaces conectadas. Se usan en transmisiones largas.

5.6.2 Topología Bus (Obsoleta)

Esta fue una de las primeras configuraciones utilizadas en redes Ethernet.

Consiste en un **bus único** de cable coaxial al que se conectan los adaptadores mediante conectores **T**. En cada extremo del bus, hay **terminadores** para evitar reflexiones de señal. Los adaptadores transmiten y reciben señales eléctricas a través del cable.

Soporta un número limitado de adaptadores por segmento. No permite más de **4 repetidores** entre [extremos](#). La **distancia máxima** está definida por la especificación del estándar utilizado.

Nomenclatura: El formato de las denominaciones sigue este patrón: **Mbps, tipo de transmisión, centenas de metros**: 10Base2 :

- Velocidad de 10 Mbps.
- Banda base (sin modulación).
- Segmentos de hasta 200 metros.

5.6.3 Topología Estrella

Es la **configuración predominante en redes Ethernet modernas**. Cada nodo está conectado mediante un **par trenzado** o **fibra óptica** a un dispositivo central (*hub o conmutador | switch*). La comunicación entre nodos siempre pasa a través de este dispositivo central.

Cada nodo usa **par trenzado** (*limitado a 100m*) o **fibra óptica**. Para velocidades superiores a **1000 Mbps**, requiere **4 pares trenzados**.

Nomenclatura: **T**: Par trenzado. **F, S, L, E**: Fibra óptica.

5.6.4 Hubs (Concentradores)

Un **hub** es un dispositivo simple que también opera en la **capa física** del modelo OSI. **Regenera y retransmite los bits** que recibe por una interfaz a **todas las demás interfaces**. No distingue entre las

tramas que transmite, lo que puede generar **colisiones** si dos adaptadores transmiten simultáneamente. Han quedado obsoletos y han sido reemplazados por dispositivos más avanzados como **switches**.

5.6.5 Bridges (Puentes) y Switches (Conmutadores)

Los bridges y switches operan en las capas **física** y **de enlace**. Su objetivo principal es **procesar tramas Ethernet, filtrar tráfico y evitar colisiones**.

Los **bridges** tienen pocas interfaces (generalmente 2 o 4). Ya no se utilizan en redes modernas. Los **Switches** soportan **decenas de interfaces**. Reemplazaron a los bridges debido a su mayor capacidad.

Inspeccionan campos como la **dirección destino** para decidir el reenvío. Detectan y descartan tramas con **errores**. Gestionan el tráfico en interfaces ocupadas almacenando tramas temporalmente.

Construyen una **tabla de reenvío** dinámica. La tabla almacena la **dirección MAC**, la **interfaz asociada** y el **tiempo de última actualización**. Inicialmente, la tabla está vacía, y el dispositivo recurre a **difusión**.

Si la dirección de destino está en la tabla, **la trama se reenvía únicamente por la interfaz correspondiente, evitando colisiones y mejorando la eficiencia**. Si la interfaz origen coincide con la de destino, se filtra la trama (se elimina). Si no está en la tabla el switch utiliza **difusión** para localizar al adaptador.

5.7 VLANs (Redes de Área Local Virtuales)

Anteriormente las redes institucionales, cada **switch** formaba una red **LAN propia**, pero esto tenía varios inconvenientes:

- Si un usuario se cambia físicamente de departamento, no podía seguir conectado al anterior. Solo hay un dominio de broadcast único (*para tramas de mensajes ARP o DHCP*).
- Solo hay un dominio de broadcast único (*para tramas de mensajes ARP o DHCP*).
- Uso ineficientes de los switches (*cada uno solo tenía unos pocos puertos*).

Por lo que si en una empresa la red va **aumentando** vamos a tener que comprar más y más **switches** y es prácticamente imposible aprovechar todos los **puertos** de esta forma, por lo que además de **añadir complejidad innecesaria a la red** se añade un **costo económico inasumible** (*tener en cuenta que cada switch se va a conectar a un router y pasar ese cable de ethernet de un piso a otro del edificio es caro*).

Estos problemas se abordan con **switches** compatibles con VLANs, que soporten el estándar IEEE 802.1Q (*añade unos campos a la cabecera*). **Estos switches permiten definir múltiples LANs**

virtuales sobre una única red fija. Los host de una VLAN se comunican entre si como si solo ellos (y ningún otro host) estuvieran conectados al switch.

En una **VLAN basada en puertos**, el administrador de la red **divide los puertos (interfaces) del switch en grupos. Cada grupo constituye una VLAN.** Se mantienen una tabla de puertos - VLAN. Solo se entregan tramas **entre puertos de la misma VLAN.**

Si se produce el caso anterior de que un usuario cambia físicamente e departamento, solo hay que actualizar la tabla de puertos para que ahora otro puerto pase a formar parte del VLAN de ese departamento.

Conseguimos:

- **Aislamiento del tráfico:** solo se entregan tramas entre puertos de la misma VLAN.
- **Pertenencia dinámica:** asignación dinámica de puertos a VLANs (*se establece en que VLAN se sitúa automáticamente*). Por ejemplo, un empleado que se conecta al puerto de red en el área de finanzas podría ser asignado automáticamente a la VLAN de finanzas, mientras que un invitado conectado a la red Wi-Fi se podría asignar a una VLAN aislada para invitados.
- **Reenvío entre VLANs mediante encaminamiento:** en la práctica se combinan routers y switches.

5.8 Conmutación de etiquetas multiprotocolo (MPLS)

La conmutación de etiquetas multiprotocolo es un mecanismo de transporte de datos. Opera entre la **capa de enlace de datos** y la **capa de red** del modelo OSI. **Fue diseñado para unificar el servicio de transporte de datos** para las redes basadas en **circuitos virtuales** y en **redes de datagramas**. Puede ser utilizado para transportar diferentes tipos de tráfico.

RECODATORIO

Las redes de circuitos virtuales son un modelo de transmisión de datos, en las cuales se establece un camino lógico entre el emisor y el receptor antes de que los datos se transfieran.

MPLS busca un enfoque diferente intentando juntar la **red de circuitos virtuales** cuando sea posible (*etiquetando selectivamente los datagramas y permitiendo a los routers el reenvío de estos datagramas en base a etiquetas de longitud fija*), pero también utilizando los datagramas (*direccionamiento y encaminamiento IP*)

Así se logró mezclar de forma efectiva las técnicas de **circuitos virtuales de una red de datagramas enrutados**.

Exp: bits experimentales, relacionados con la QoS. **S:** stack, vale 1 si es la última etiqueta de la jerarquía.

Una trama ampliada MPLS solo se puede intercambiar entre routers **compatibles con MPLS**.

5.8.1 Routers de conmutación de etiquetas

Los routers compatibles con MPLS cuando reciben una trama:

- Analiza la etiqueta MPLS en vez de revisar toda la dirección IP
- Consultan una tabla de conmutación de etiquetas para decidir como reenviar el paquete.
- No tocan para nada la cabecera IP.

5.8.2 Cálculo de rutas entre routers compatibles con MPLS

Para calcular este tipo de rutas se utiliza una extensión de algoritmos como OSPF(open shortest path first). Cada fabricante utiliza el algoritmo que quiera.

5.8.3 Cálculo de rutas entre routers compatibles con MPLS

- **Ingeniería de tráfico:** El direccionamiento IP siempre nos intenta dar la ruta más corta, en cambio **MPLS proporciona la capacidad de reenviar** paquetes a **través de rutas** que no serían posibles utilizando los protocolos de enrutamiento IP estándar. **Se puede anular en enrutamiento IP normal y forzar a que aparte del tráfico dirigido hacia un cierto destino tome una determinada ruta**, mientras que el resto del tráfico dirigido siga **rutas distintas**.
- **Establecer VPNs**
- Aislar tanto los recursos como el direccionamiento empleados por la **VPN** del cliente con respecto a los de otros usuarios que también tenga que atravesar la red del ISP.

5.9 WLAN

WLAN significa **red de área local inalámbrica** y se basa en el estándar IEEE 802.11, que define cómo se comunican los dispositivos inalámbricos (como routers, móviles, laptops, etc.) usando Wi-Fi.

El estándar 802.11 tiene varias versiones (b, a, g, n, etc.), y cada una mejora aspectos como la velocidad, el alcance, y la resistencia a interferencias.

5.9.1 Redes Ad-hoc

Se crean directamente entre dispositivos sin necesidad de un punto de acceso (Access Point, AP). Son redes **temporales** y de **igual a igual** (peer-to-peer). Solo funcionan mientras los dispositivos están **dentro del radio de alcance**.

5.9.2 Redes distribuidas (Managed)

Utilizan puntos de acceso (**Access Points, AP**) conectados a una **LAN cableada**. AP conecta a los dispositivos inalámbricos dentro de su radio y los comunica con la LAN troncal. Se usan en redes Wi-Fi gestionadas, como en **oficinas o en casa** con routers Wi-Fi.

5.9.3 Protocolo de Acceso al Medio (MACA)

MACA (**Multiple Access with Collision Avoidance**) o **CSMA/CA** es el protocolo que evita colisiones en redes inalámbricas (donde no se pueden detectar colisiones como en Ethernet). Así funciona:

Un dispositivo que quiere transmitir verifica si el canal está [libre](#). Si el canal está libre, espera un intervalo llamado **DIFS** (Distributed Inter Frame Space) para asegurarse de que sigue libre. Si aún está libre, comienza a transmitir. Si el canal está ocupado, sigue esperando y usa un mecanismo de **backoff exponencial** donde el dispositivo espera un tiempo aleatorio creciente antes de intentarlo de nuevo.

No puede detectar colisiones directamente, así que confía en los **ACKs**, después de transmitir, el emisor espera un **ACK** (acuse de recibo) del receptor. Si no recibe el ACK, supone que hubo un problema y **MACA / CSMA/CA (Protocolo de acceso al medio)**.

5.9.4 Ejemplo Práctico

Supón que tienes un router Wi-Fi en casa y dispositivos como una ordenador y un móvil:

1. El móvil quiere enviar datos:

- Verifica si el canal está libre. Si lo está, envía los datos tras el intervalo DIFS.
- Si el canal está ocupado, espera y reintenta según el algoritmo de backoff.

2. El router confirma la recepción:

- Responde con un **ACK** si recibió los datos correctamente.
- Si el móvil no recibe el ACK, reintenta.

3. Otros dispositivos respetan las tramas RTS/CTS:

- Si el móvil y el router están transmitiendo, los demás dispositivos en la red esperan hasta que acaben.

5.10 Redes ATM

Es un tipo de red con el que **trabajan las compañías telefónicas**. Fue diseñada para operar a **alta velocidad**. Pueden transmitir **datos, voz y video**. Los **conmutadores** pueden operar a velocidades de

terabit por segundo. El modelo ATM cubre las **tres capas inferiores**:

- **Capa física**
- **Capa de enlace**
- **Capa de red**

Se integra en la **arquitectura TCP/IP**. Se usaba en **redes telefónicas** y en las **troncales de Internet**.

5.10.1 Tipos de servicio en ATM

CBR (Constant Bit Rate): Se reserva y garantiza una cierta **tasa de transmisión**. Los **retardos y las pérdidas** están bajo ciertos límites garantizados. Adecuado para transmitir **audio y vídeo**.

ABR (Available Bit Rate): La **tasa de transmisión varía** según los recursos disponibles, pero se garantiza un mínimo. NO se garantiza un mínimo en las **pérdidas o el retardo**.

UBR (Unspecified Bit Rate): Solo se transmiten paquetes cuando el resto de los servicios dejan recursos libres.

VBR (Variable Bit Rate): Se usa para aplicaciones en tiempo real (*VBR-rt*) o no en tiempo real (*VBR-nrt*).

5.10.2 Características de las redes ATM

Paquetes **muy pequeños y sencillos** (*celdas*) para garantizar su **conmutación a altas velocidades**. Tamaño de celda de **53 bytes** (*5 de cabecera y 48 de datos*).

Red de **circuitos virtuales** orientada a conexión. Antes de la transmisión, hay **una solicitud de conexión**. Se planifica la **ruta**. Las celdas llevan el **número de canal virtual**. El **conmutador ATM** consulta la tabla de canales virtuales y selecciona la línea de salida. Todas las celdas siguen **el mismo camino** y llegan **en orden**. Al finalizar hay una fase de **desconexión** donde se elimina los canales virtuales.

No hay ACKs ni **retransmisiones**, pero las celdas tienen **control de errores de la cabecera**

5.10.3 Capas de ATM

ATM puede funcionar sobre **cualquier capa física**. La **capa de adaptación a ATM (AAL)** permite que otros protocolos usen la red ATM. Diferentes AAL dependiendo del tipo de servicio:

- **TCP/IP**: Los datagramas se fragmentan para que quepan en las celdas y se reensamblan a la salida.
- **Audio y vídeo**: se agrupan los datos hasta llenar una celda.

5.10.4 Identificación de circuitos virtuales**

En ATM hay dos niveles de conexión:

- **VCC**(*Virtual Channel Connection*): Canal virtual.
- **VPC**(*Virtual Patch Connection*): Camino virtual.
 - Conjunto de **VCCs** con los mismos extremos.
 - Facilitan la gestión de los **VCCs**.

El identificador de circuito virtual permite distinguir entre caminos virtuales, y dentro de cada camino virtual entre canales.

5.10.5 Estructura de las celdas ATM

Dos formatos:

- **Interfaz usuario-red**
- **Interfaz red-red**

Control de Flujo Genérico (GFC): solo en las interfaz usuario-red para la QoS. **Identificador de canal virtual**: VPI y VCI **Tipo de carga útil**: si la celda es de **datos** o de **control** **Bit de prioridad**: de la celda (**CLP**) **Byte de control de error** de la cabecera.

5.10.6 Ejemplo práctico de ATM

Imagina que un proveedor de servicios de Internet (ISP) usa ATM para transmitir tráfico de red:

1. Un cliente envía datos desde su casa.
2. El ISP utiliza ATM para dividir esos datos en celdas pequeñas (53 bytes) y las envía a través de su red troncal.
3. Las celdas siguen un circuito virtual preestablecido, identificado por VPI y VCI.
4. Cuando llegan al destino, las celdas se reensamblan para reconstruir los datos originales.